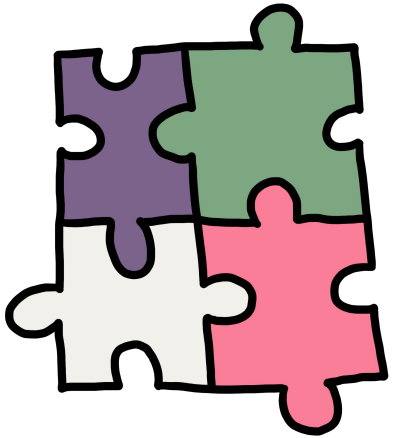
Research Software Engineering

DAY 1

Dr. Fritjof Lammers

2-day workshop



Day 1

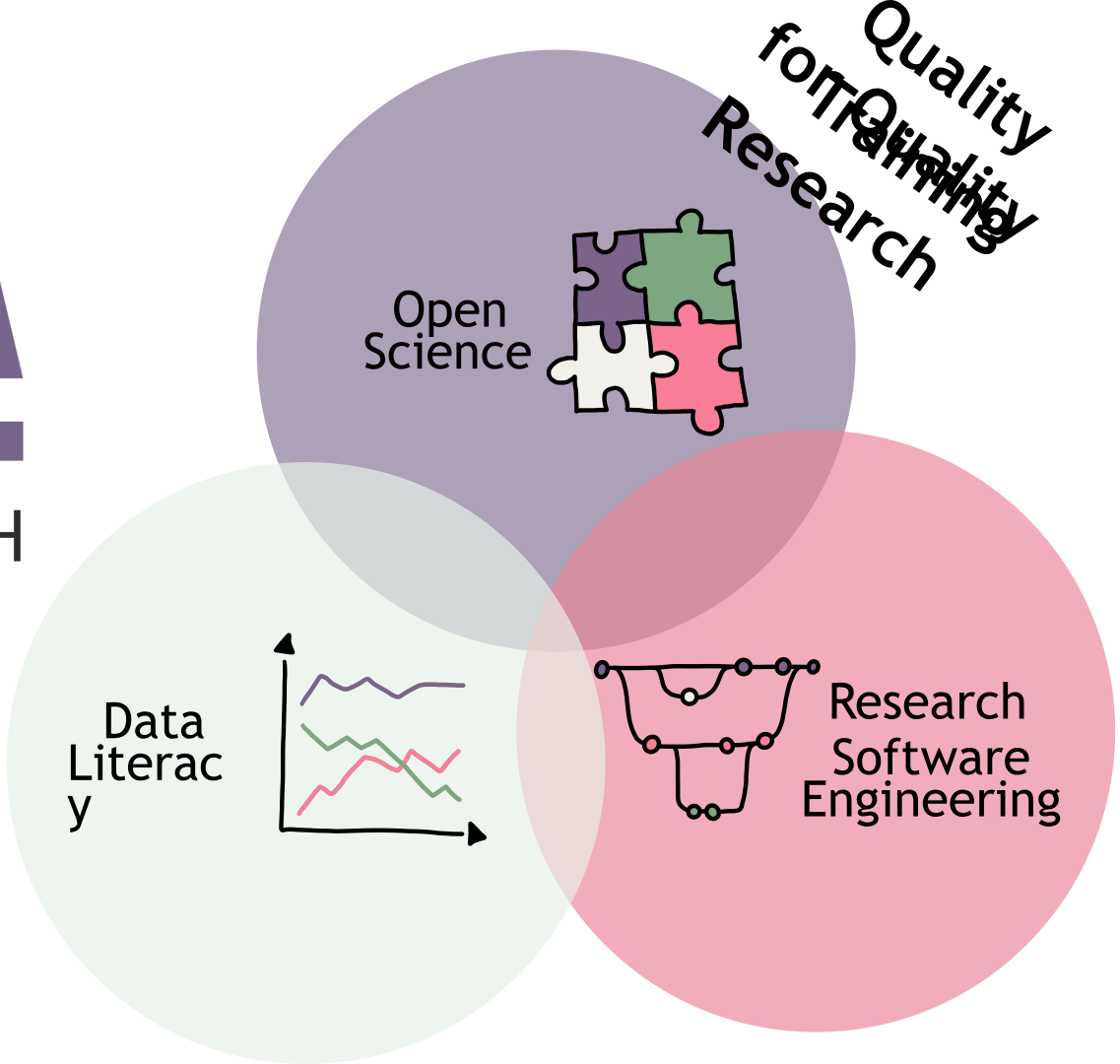
- What is Research Software Engineering
- Collaborative coding with Git & Github
- Reproducible coding

Day 2

- Introduction to testing
- Continuous Integration and Delivery (CI/CD)
- Software publication and licensing

DRA

DIGITAL RESEARCH ACADEMY



Code of Conduct

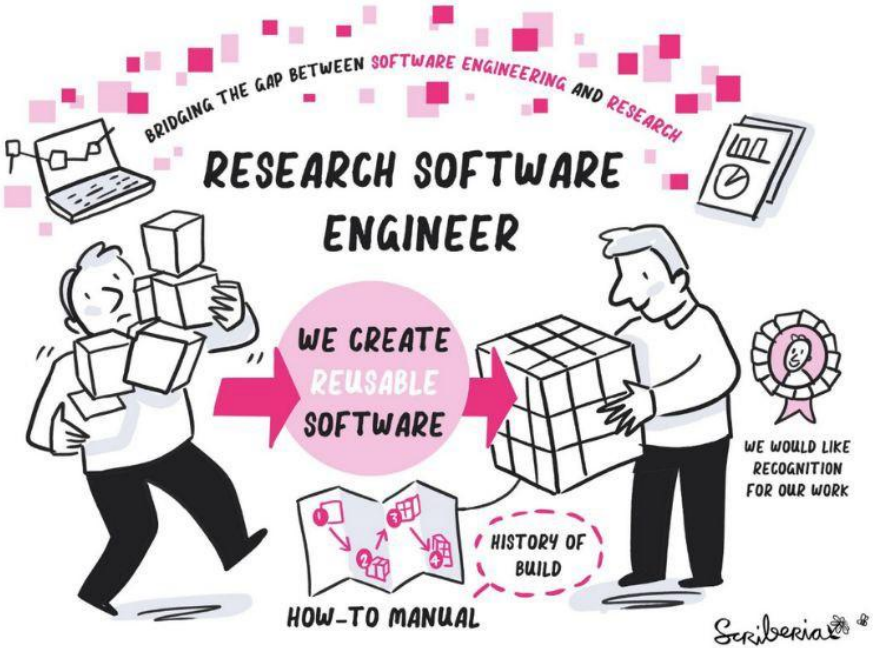
Everyone is welcome, make everyone feel welcome, be nice.

https://digital-research.academy/code-of-conduct/


Please ask questions
https://pad.okfn.de/p/rse-fzj-202511

Who are you and why are you here?

What is a Research Software Engineer?



Discussion 1

What is a Research Software Engineer?

What is a Research Software Engineer?

Definition is quite imprecise and notes from the original meeting where the term was coined (UK, 2012), include more a description of what it NOT is:

Roles: “research officer” – considered support don’t want to be servants

NOT permanent postdoc

Not a lab technician
Calling them software engineers is not good enough – domain specialised

“Researcher” has pressure to publish

https://www.software.ac.uk/blog/not-so-brief-history-research-software-engineers-0

What is a Research Software Engineer?

… t**he research software engineer**. These individuals combine a *professional attitude* to the exercise of software engineering with a *deep understanding of research topics*.

*Computational work must reflect the committed attitude of experimentalists towards caring about precise, professional, repeatable, meticulous work - **no-one with the same casual attitude to experimental instrumentation as many researchers have to code would be allowed anywhere near a lab**. This is striking considering how often research results now depend on software.*

Baxter et al. 2012

What is a Research Software Engineer?

F1000Research F1000Research 2024, 13:1429 Last updated: 27 NOV 2024

OPINION ARTICLE

Foundational Competencies and Responsibilities of a Research Software Engineer

[version 1; peer review: awaiting peer review]

Florian Goth[1], Renato Alves[2], Matthias Braun[3], Leyla Jael Castro[4], Gerasimos Chourdakis[5,6], Simon Christ[7], Jeremy Cohen[8], Stephan Druskat[9], Fredo Erxleben[10], Jean-Noël Grad[11], Magnus Hagdorn[12], Toby Hodges[13], Guido Juckeland[10], Dominic Kempf[14], Anna-Lena Lamprecht[15], Jan Linxweiler[16], Frank Löffler[17], Michele Martone[18], Moritz Schwarzmeier[19], Heidi Seibold[20], Jan Philipp Thiele[21,22], Harald von Waldow[23], Samantha Wittke[24]

What is a Research Software Engineer?

Training/teaching

Work in research environment

Produce software

Communicators/Translators

self-identify

Produce papers

Research Software Engineer

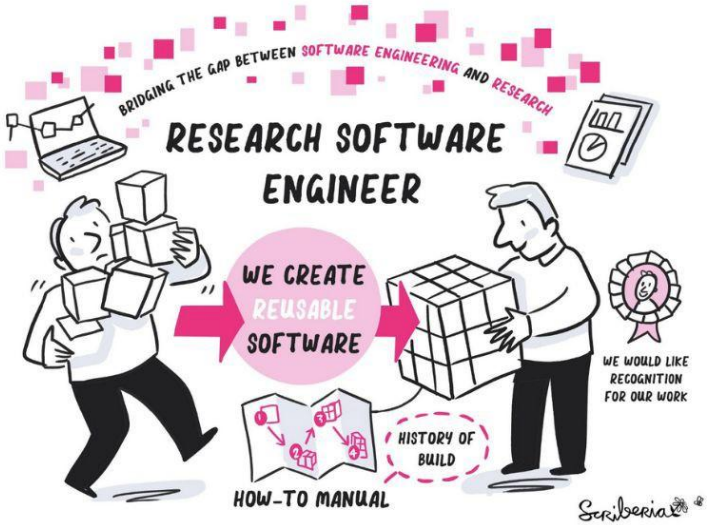
funding

Research

Software Developer

What is Research Software

Foundational algorithms, the software itself, as well as scripts and computational workflows that were **created during the research process or for a research purpose**, across all domains of research.



Setting up a (research) code project

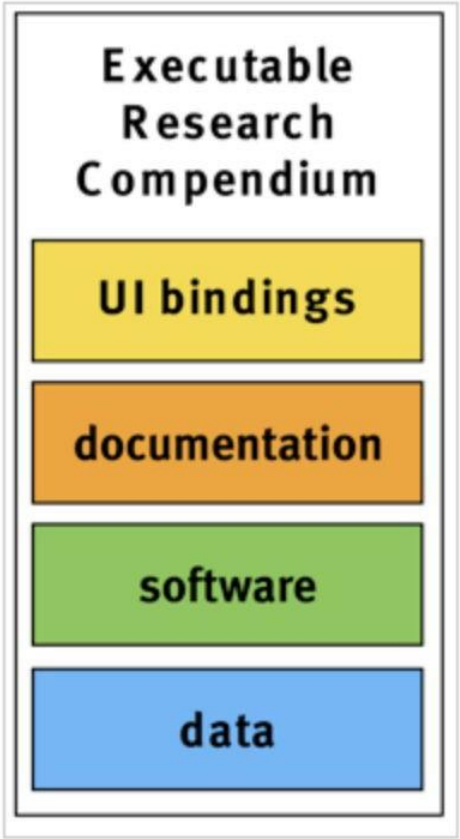
DIGITAL RESEARCH
ACADEMY

The Research compendium

The Research Compendium

- A research compendium is a collection of all digital parts of a research project including data, code, texts (protocols, reports, questionnaires, meta data). **The collection is created in such a way that reproducing all results is straightforward.**

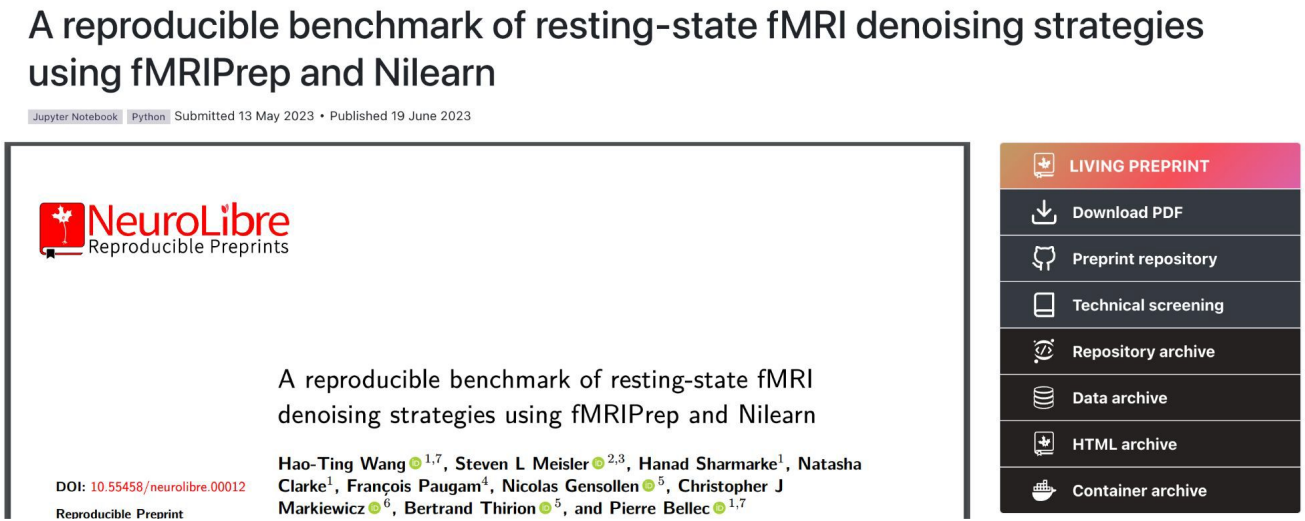
- A research compendium accompanies, enhances, or is a scientific publication providing data, code, and documentation for reproducing a scientific workflow.



Nuest, D., Konkol, M., Pebesma, E., Kray, C., Schutzeichel, M., Przibytzin, H., & Lorenz, J. (2017). Opening the Publication Process with Executable Research Compendia. *D-Lib Magazine*, *23*(1/2). 10.1045/january2017-nuest

Neurolibre

- FULLY reproducible paper
- Living preprint: reviewer comments are visible; interactive document

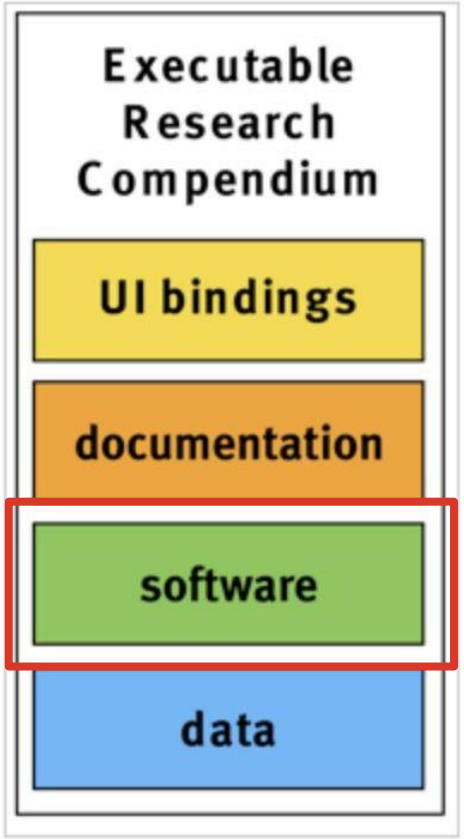


The Research Compendium

A very comprehensive website on research compendia can be found here

An example template to document your research can be found here

Reproducible science includes a good organisation of the software/code



Setting up a code project - as part of a research compendium

The folder structure

```
compendium/
├── data
│   ├── my_data.csv
├── analysis
│   └── my_script.R
├── DESCRIPTION
└── README.md
```

```
compendium/
├── CITATION
├── code
│   ├── analyse_data.R
│   └── clean_data.R
├── data_clean
│   └── data_clean.csv
├── data_raw
│   ├── datapackage.json
│   └── data_raw.csv
├── Dockerfile
├── figures
│   └── flow_chart.jpeg
├── LICENSE
├── Makefile
├── paper.Rmd
└── README.md
```

https://the-turing-way.netlify.app/reproducible-research/compendia.html

Separate Methods, Data, Output

- **Read-only**: raw data (`data_raw\`), metadata (`datapackage.json`, `CITATION`)
- **Human-generated**: code (`clean_data.R`, `analyse_data.R`), paper (`paper.Rmd`), documentation (`README.md`)
- **Project-generated**: clean data (`data_clean\`, figures (`figures\`), other output

Setting up a code project

```
my-research-project/
├── .vscode/           # vs code settings
│   ├── launch.json         # Cleaned, derived data
│   └── settings.json        # Data documentation
├── src/                # Code (modules, scripts)
├── notebooks/           # Jupyter or MATLAB Live Scripts
├── data/
│   ├── raw/             # Untouched raw data
│   ├── processed/         # Cleaned, derived data
│   └── README.md         # Data documentation
├── results/
│   ├── figures/          # Plots/images generated
│   └── tables/           # CSV, LaTeX tables, etc.
├── paper/
│   ├── manuscript.md          # Or .tex, .docx,
etc.├── bibliography.bib # References
│   └── figures/   # Final figures for the paper
├── tests/               # Automated tests
├── requirements.txt       # or environment.yml / setup.py
├── Makefile             # Automate workflows (optional)
├── LICENSE
├── README.md       # What is your project about?
└── .gitignore
```

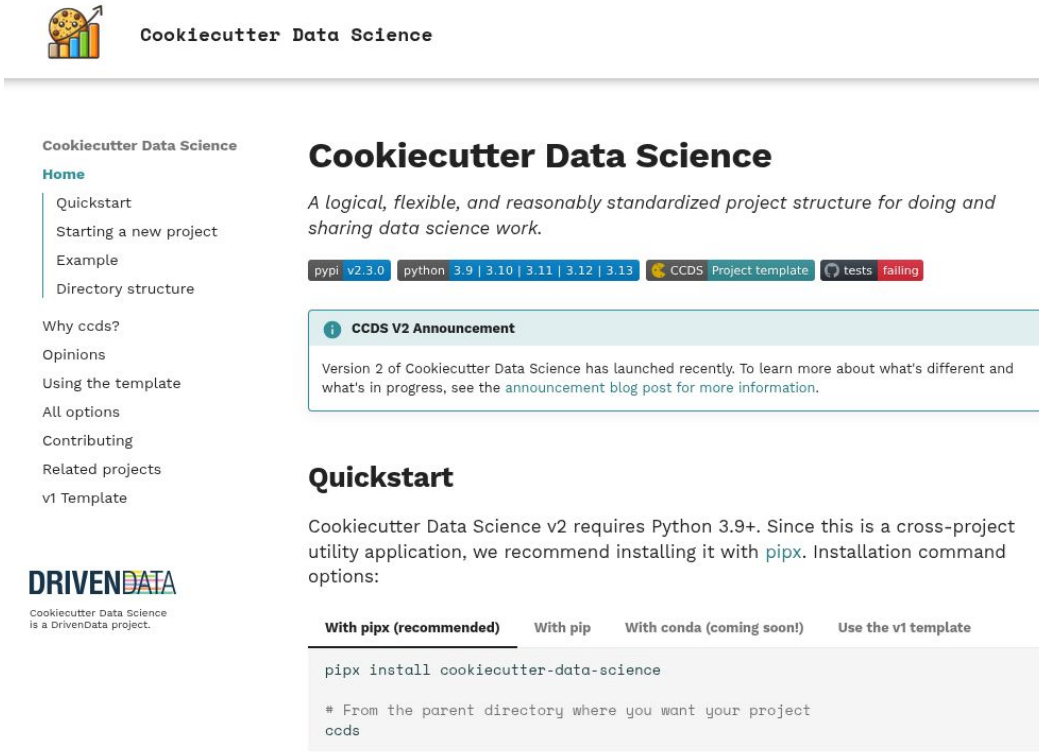
Cookiecutter project template

- Code and data version for good file organisation

```
johannabayer@Hanna my_project % tree
.
├── LICENSE
├── Makefile
├── README.md
├── data
│   ├── external
│   ├── interim
│   ├── processed
│   └── raw
├── docs
├── models
├── my_project
│   ├── __init__.py
│   ├── config.py
│   ├── dataset.py
│   ├── features.py
│   ├── modeling
│   │   ├── __init__.py
│   │   ├── predict.py
│   │   └── train.py
│   └── plots.py
├── notebooks
├── pyproject.toml
├── references
├── reports
│   └── figures
├── requirements.txt
└── setup.cfg
```

Project folder structure

- Cookie cutter project templates
- Cookie cutter data science (ccds)
  - Python-based project structure
- Can be a start if you more advanced



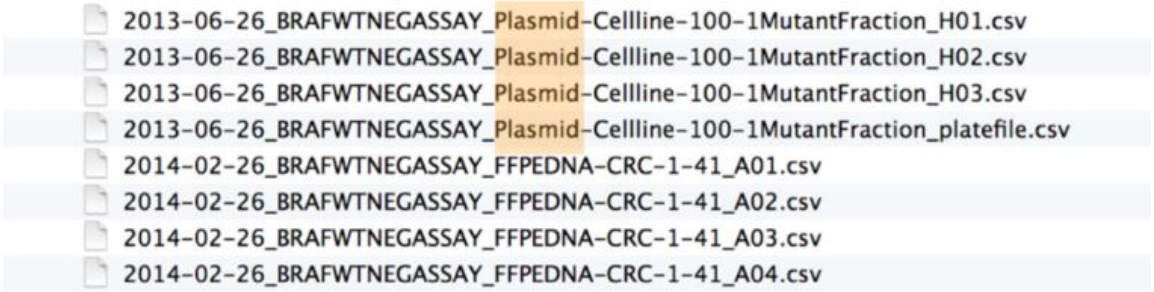
https://cookiecutter.readthedocs.io/en/stable/

Setting up a coding project - naming files

- Files need to be
  - Machine readable
    - friendly to regular expressions and globbing (*)
    - avoid spaces, punctuation, accents, case sensitive
  - Human readable
    - info on content, descriptive
    - use “slug” (a.k.a. from semantic URLs)
  - Consistent
  - Optional: Play with default ordering (i/e start your file with creation date
    - YYYY-MM-DD).

Machine readable

Excerpt of complete file listing:



Example of **globbing** to narrow file listing:

```
Jennifers-MacBook-Pro-3:2014-03-21 jenny$ ls *Plasmid*
2013-06-26_BRAFWTNEGASSAY_Plasmid-Cellline-100-1MutantFraction_A01.csv
2013-06-26_BRAFWTNEGASSAY_Plasmid-Cellline-100-1MutantFraction_A02.csv
2013-06-26_BRAFWTNEGASSAY_Plasmid-Cellline-100-1MutantFraction_A03.csv
2013-06-26_BRAFWTNEGASSAY_Plasmid-Cellline-100-1MutantFraction_B01.csv
....
2013-06-26_BRAFWTNEGASSAY_Plasmid-Cellline-100-1MutantFraction_H03.csv
2013-06-26_BRAFWTNEGASSAY_Plasmid-Cellline-100-1MutantFraction_platefile.csv
```

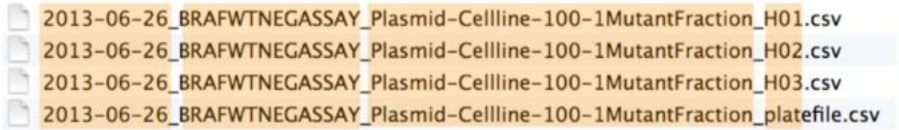
Machine readable

Same using R's ability to narrow file list by regex:

```
> list.files(pattern = "Plasmid") %>% head
[1] "2013-06-26_BRAFWTNEGASSAY_Plasmid-Cellline-100-1MutantFraction_A01.csv"
[2] "2013-06-26_BRAFWTNEGASSAY_Plasmid-Cellline-100-1MutantFraction_A02.csv"
[3] "2013-06-26_BRAFWTNEGASSAY_Plasmid-Cellline-100-1MutantFraction_A03.csv"
[4] "2013-06-26_BRAFWTNEGASSAY_Plasmid-Cellline-100-1MutantFraction_B01.csv"
[5] "2013-06-26_BRAFWTNEGASSAY_Plasmid-Cellline-100-1MutantFraction_B02.csv"
[6] "2013-06-26_BRAFWTNEGASSAY_Plasmid-Cellline-100-1MutantFraction_B03.csv"
```

Machine readable

Deliberate use of "_" and "-" allows us to recover meta-data from the filenames.




```
> flist <- list.files(pattern = "Plasmid") %>% head

> stringr::str_split_fixed(flist, "[_\\.]", 5)
     [,1]         [,2]          [,3]                                    [,4]  [,5]
[1,] "2013-06-26" "BRAFWTNEGASSAY" "Plasmid-Cellline-100-1MutantFraction" "A01" "csv"
[2,] "2013-06-26" "BRAFWTNEGASSAY" "Plasmid-Cellline-100-1MutantFraction" "A02" "csv"
[3,] "2013-06-26" "BRAFWTNEGASSAY" "Plasmid-Cellline-100-1MutantFraction" "A03" "csv"
[4,] "2013-06-26" "BRAFWTNEGASSAY" "Plasmid-Cellline-100-1MutantFraction" "B01" "csv"
[5,] "2013-06-26" "BRAFWTNEGASSAY" "Plasmid-Cellline-100-1MutantFraction" "B02" "csv"
[6,] "2013-06-26" "BRAFWTNEGASSAY" "Plasmid-Cellline-100-1MutantFraction" "B03" "csv"
```

date assay sample set well

This happens to be R but also possible in the shell, Python, etc.

“human readable”

```
Jennifers-MacBook-Pro-3:analysis jenny$ ls -1
```

```
01_marshal-data.md
01_marshal-data.r
02_pre-dea-filtering.md
02_pre-dea-filtering.r
03_dea-with-limma-voom.md
03_dea-with-limma-voom.r
04_explore-dea-results.md
04_explore-dea-results.r
90_limma-model-term-name-fiasco.md
90_limma-model-term-name-fiasco.r
Makefile
figure
helper01_load-counts.r
helper02_load-exp-des.r
helper03_load-focus-statinf.r
helper04_extract-and-tidy.r
tmp.txt
```

```
01.md
01.r
02.md
02.r
03.md
03.r
04.md
04.r
90.md
90.r
Makefile
figure
helper01.r
helper02.r
helper03.r
helper04.r
tmp.txt
```

Which set of file(name)s do you want at 3a.m. before a deadline?

“human readable”

```
01_marshal-data.r

02_pre-dea-filtering.r

03_dea-with-limma-voom.r

04_explore-dea-results.r

90_limma-model-term-name-fiasco.r

helper01_load-counts.r

helper02_load-exp-des.r

helper03_load-focus-statinf.r

helper04_extract-and-tidy.r
```

embrace the **slug**

“plays well with default ordering”

```
2013-06-26_BRAFWTNEGASSAY_Plasmid-Cellline-100-1MutantFraction_H01.csv
2013-06-26_BRAFWTNEGASSAY_Plasmid-Cellline-100-1MutantFraction_H02.csv
2013-06-26_BRAFWTNEGASSAY_Plasmid-Cellline-100-1MutantFraction_H03.csv
2013-06-26_BRAFWTNEGASSAY_Plasmid-Cellline-100-1MutantFraction_platefile.csv
2014-02-26_BRAFWTNEGASSAY_FFPEDNA-CRC-1-41_A01.csv
2014-02-26_BRAFWTNEGASSAY_FFPEDNA-CRC-1-41_A02.csv
2014-02-26_BRAFWTNEGASSAY_FFPEDNA-CRC-1-41_A03.csv
2014-02-26_BRAFWTNEGASSAY_FFPEDNA-CRC-1-41_A04.csv
```

```
01_marshal-data.r
02_pre-dea-filtering.r
03_dea-with-limma-voom.r
04_explore-dea-results.r
90_limma-model-term-name-fiasco.r
helper01_load-counts.r
helper02_load-exp-des.r
helper03_load-focus-statinf.r
helper04_extract-and-tidy.r
```

put something numeric first

Naming files

(BAD) Examples:

- Myabstract.docx
- Joe’s Filenames Use Spaces and Punctuation.xlsx
- figure 1.png
- fig 2.png
- JW7d^(2sl@deletethisandyourcareerisoverWx2*.tx
  t

Naming files

Better examples

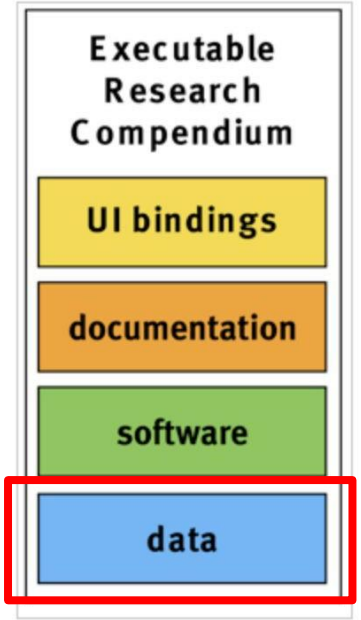
- YYYYMMDD_OHBM_abstract.txt
- Step_1_step_description
- Step_2_step_description
- Unique filenames:
  - Sub_1
    - Sub_1_fileA.txt
    - Sub_1_fileB.txt
  - Sub_2
    - sub_2_fileA.txt
    - sub_2_fileB.txt

Follow the software example: versioning

- Generic version is the CURRENT/IMPORTANT version (eg. abstract.txt)
- All older/other versions are versioned (e.g abstract_Sept_2024.txt)
- Similar to JAVA/Git - > main method/master is the one that compiles, not main_2025, even if that's the latest version.h

https://berd-nfdi.github.io/BERD-reproducible-research-course/2-1-naming.html

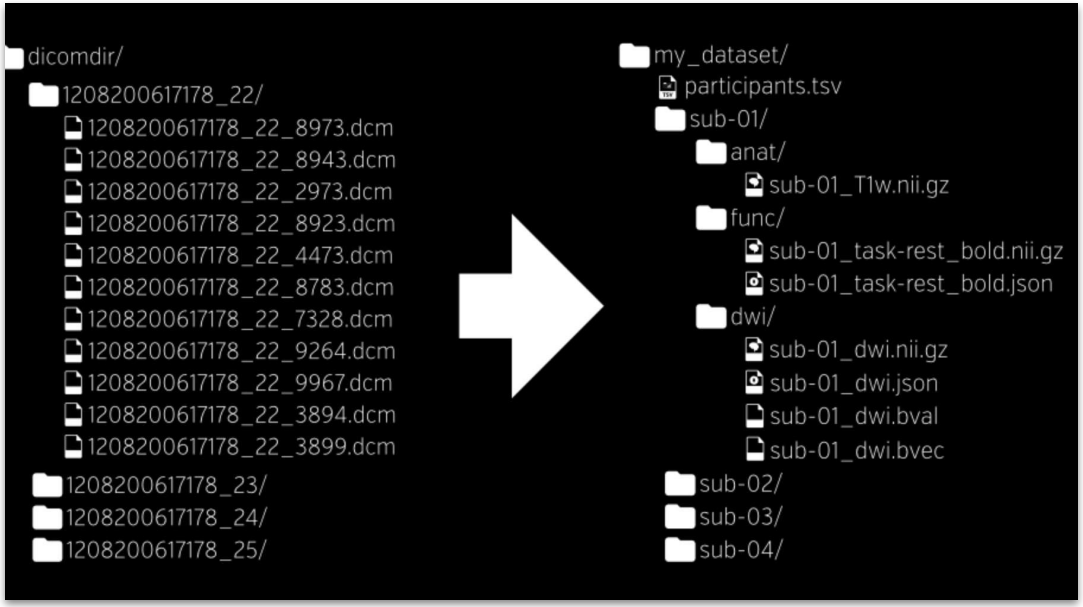
Data organisation - does your field have a file standard?




```
my-research-project/
├── .vscode/          # vs code settings
│   ├── launch.json       # Cleaned, derived data
│   └── settings.json     # Data documentation
├── src/              # Code (modules, scripts)
├── notebooks/        # Jupyter or MATLAB Live Scripts
├── data/
│   ├── raw/          # Untouched raw data
│   ├── processed/    # Cleaned, derived data
│   └── README.md         # Data documentation
├── results/
│   ├── figures/      # Plots/images generated
│   └── tables/       # CSV, LaTeX tables, etc.
├── paper/
│   ├── manuscript.md     # Or .tex, .docx, etc.
│   ├── bibliography.bib  # References
│   └── figures/      # Final figures for the paper
├── tests/            # Automated tests
├── requirements.txt      # or environment.yml / setup.py
├── Makefile          # Automate workflows (optional)
├── LICENSE
├── README.md         # What is your project about?
└── .gitignore
```

Does your field have a file standard?

Brain Imaging Derived Structures



(https://bids.neuroimaging.io/);

Slides by courtesy of Dr. Remi Gau, adapted

File organisation: (stealing from the) BIDS standard

Brain Imaging Derived Structures

- modularizes data
- names files in a human AND machine friendly way
- specifies a folder structure
- uses standard interoperable file formats
- documents metadata
- minimize duplication

Slides by courtesy of Dr. Remi Gau, adapted

File organisation: (stealing from the) BIDS standard

Brain Imaging Derived Structures

Key1-value_key2-value_suffix.extension

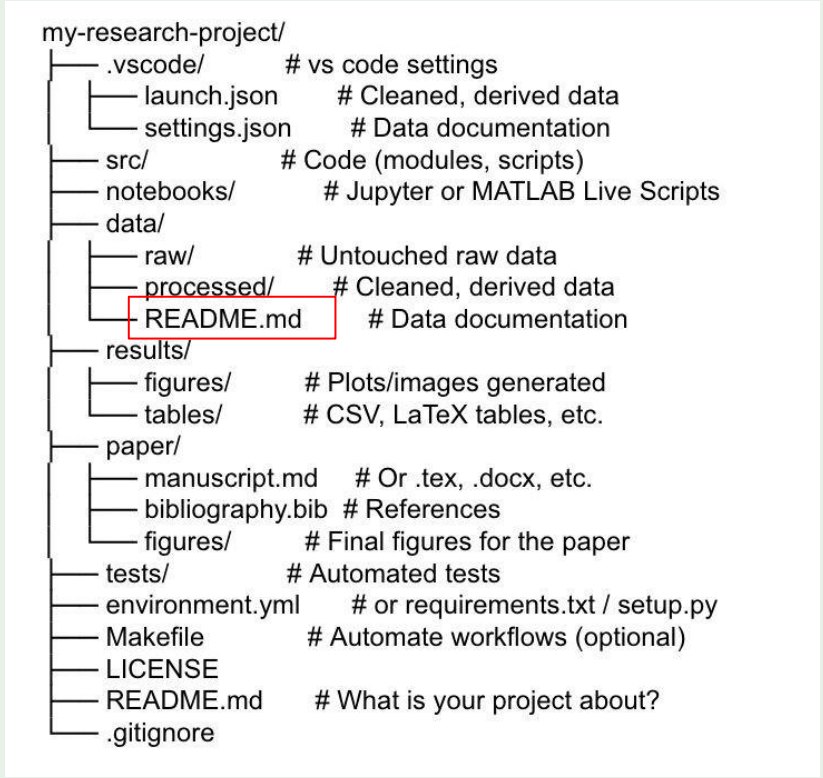
sub-001_population-cross_mri.nii

- **Suffix** preceded by an **underscore**
- Entities are composed of **key-label** pairs separated by **underscores**
- **Key** and **label** separated by **hyphen**
- **Keys**, **labels**, **suffixes** can only contain letters and / or numbers.
- Key-value combination can be repetitions from higher order folders to form unique filenames

Slides by courtesy of Dr. Remi Gau,
adapted

Adding a Readme.md

- Entry point/first item that visitor reads
- Follows Markdown format
- There are several (slightly differing) markdown formats, but we are going to focus on git markdown.



Readme.md

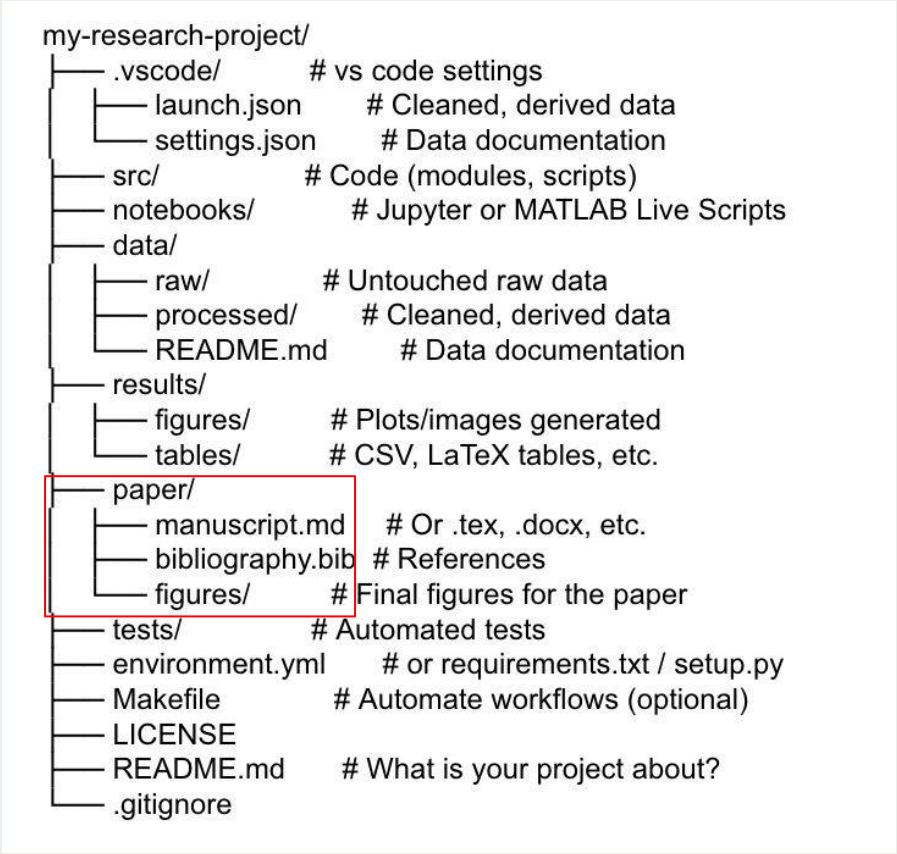
Files ending in .md indicate the Markdown style. We are going to follow Github Markdown.

A Readme.md should contain:

- Title and description of the project
- Author of the project and contact details
- Information about the license of the project - can everyone reuse your code?
- Structure of the project
- Anything that a user of the project should know

Examples: pydantic

Thinking about the writing early on



Text editor and reference management system (based on reproducibility criteria)

Text editor	pro	con
MS Word/Google docs	-Plugins for most reference managers -good (?) platform for getting feedback/review -(Version controlled)	- Figures need to be imported each time
Markdown	-Integration with R (markdown, knitr) -In theory, fully reproducible (code and paper)	- No review option
Overleaf (online)	-Bibtex/Latex -Pro version has direct git and Dropbox link -Math support, templates -review option	- steeper learning curve
LaTeX (Desktop)	-Free -Bibtex file -Entirely version controllable -Math support -Create a figure folder that you save your figs in	-Very steep learning curve -No review option

Collaborative publishing using overleaf

- All benefits of pure latex but
  - Working on a document together & review mode (suggest changes)
  - Import citations from paperpile, zotero, mendeley and endnote
  - Templates for publications, journals, conference posters etc
  - Link an image folder (i.e in Dropbox)
  - Git integration

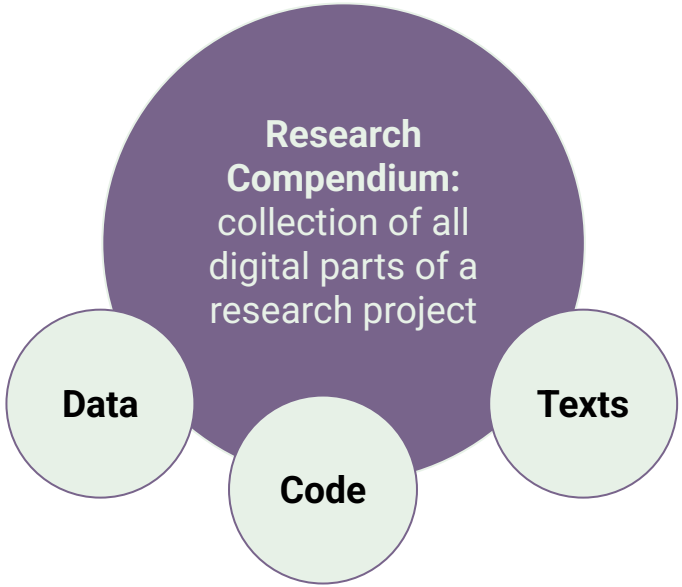
Downside

- Internet dependency (except offline mode)

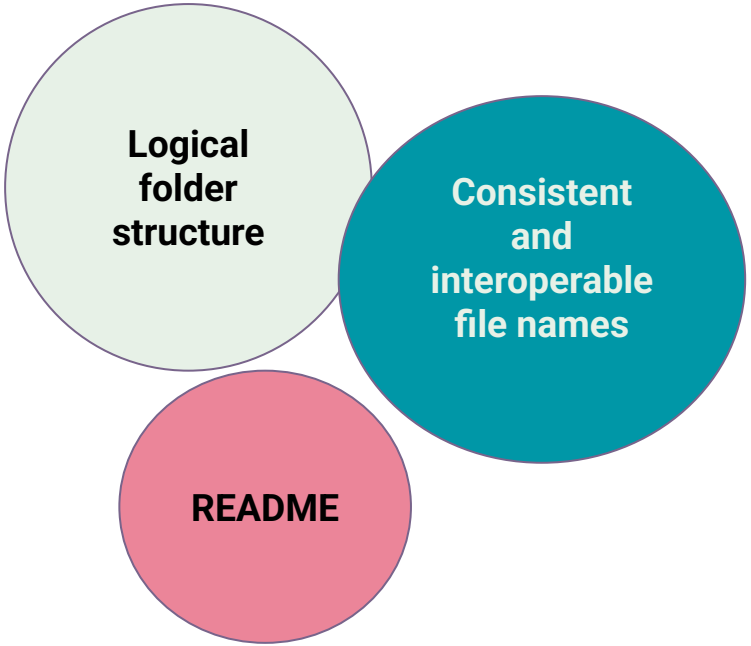
More resources on Research Compendia

- https://github.com/topics/research-compendium/
- https://the-turing-way.netlify.app/reproducible-research/compendia
- https://research-compendium.science/
- https://selinazitrone.github.io/YoMos2020/slidy_presentation.html#(1)

Summary



The collection is created in such a way that reproducing all results is straightforward

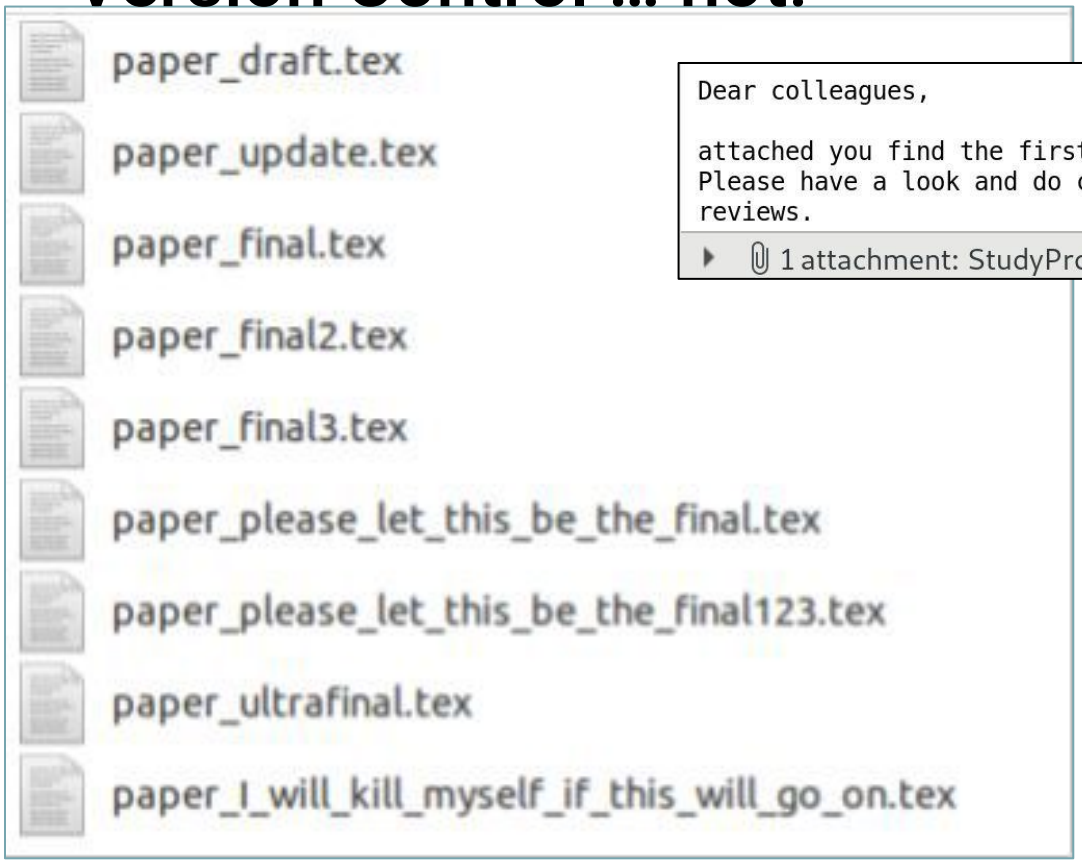


Break

Version control using Github/Gitlab

Community, T. T. W., & Scriberia. (2020). *Illustrations from the Turing Way book dashes*. Zenodo. 10.5281/ZENODO.3695300

Version Control … not!

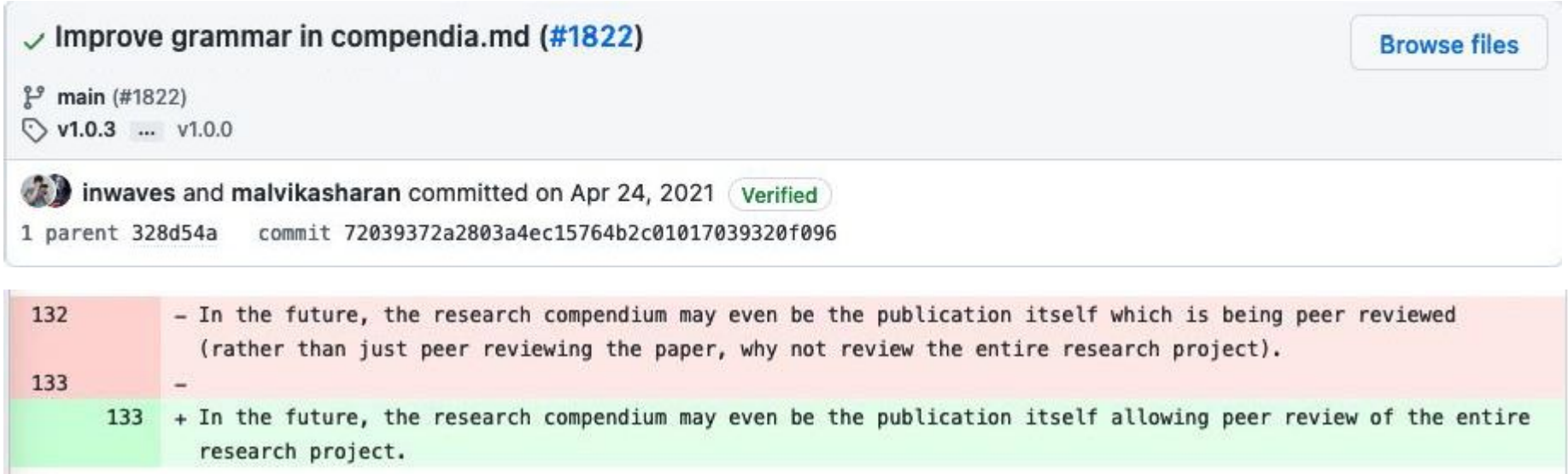


Dear colleagues,

attached you find the first public version of the [redacted] protocol. Please have a look and do comment. We can also meet to aggregate our reviews.

1 attachment: StudyProposa[redacted]_Validation_V1_250918docx.docx

Version Control: Git as a solution



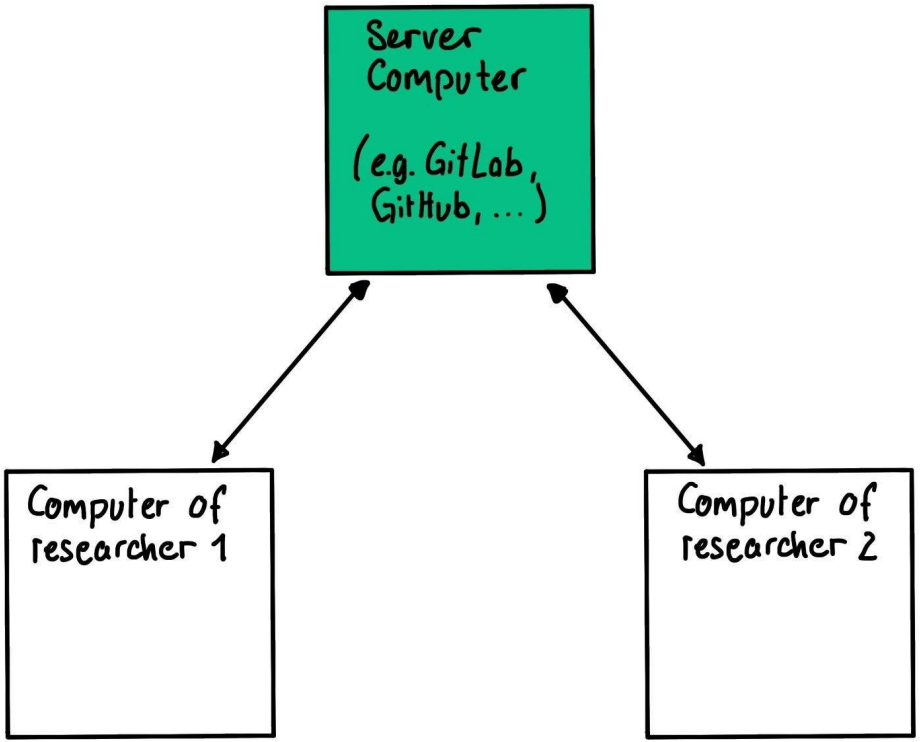
Screenshot:
https://github.com/alan-turing-institute/the-turing-way/commit/72039372a2803a4ec15764b2c01017039320f0

How does Version Control work?

*Track **who** made **which** change and **when***

- Track different versions of your code, text, …
- Description of each version (“commit message”)

Gi
t


Server Computer
(e.g. GitLab, GitHub, ...)
Computer of researcher 1
Computer of researcher 2

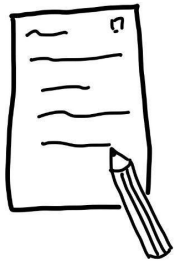
Git

Software on your machine

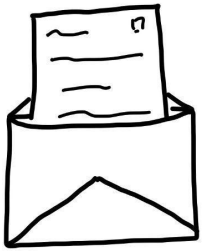
GitLab / GitHub

Service to use as remote server

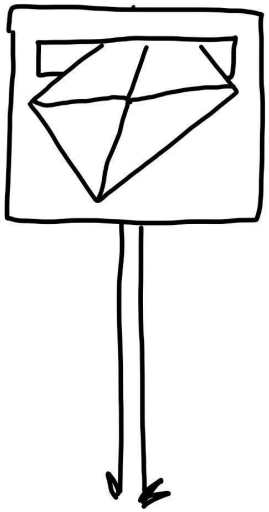
Git commands for creating changes



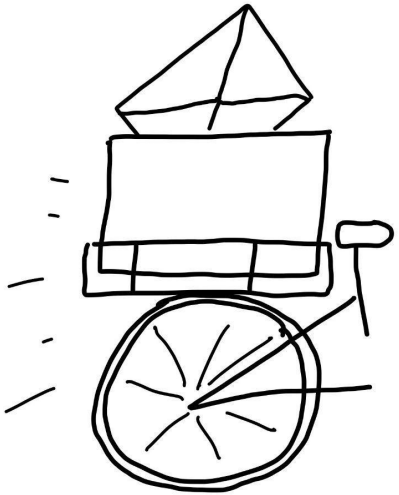
edit document



add



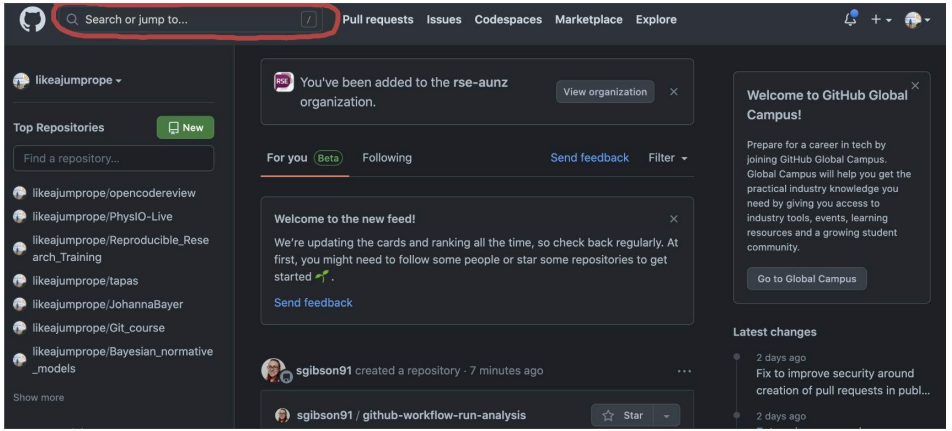
commit



push

Getting to know the Github interface

1. Log into your github account
2. The login page will get you to your dashboard. Here you find:
   a. An overview of your dashboard
   b. Your profile
   c. An overview of your repositories

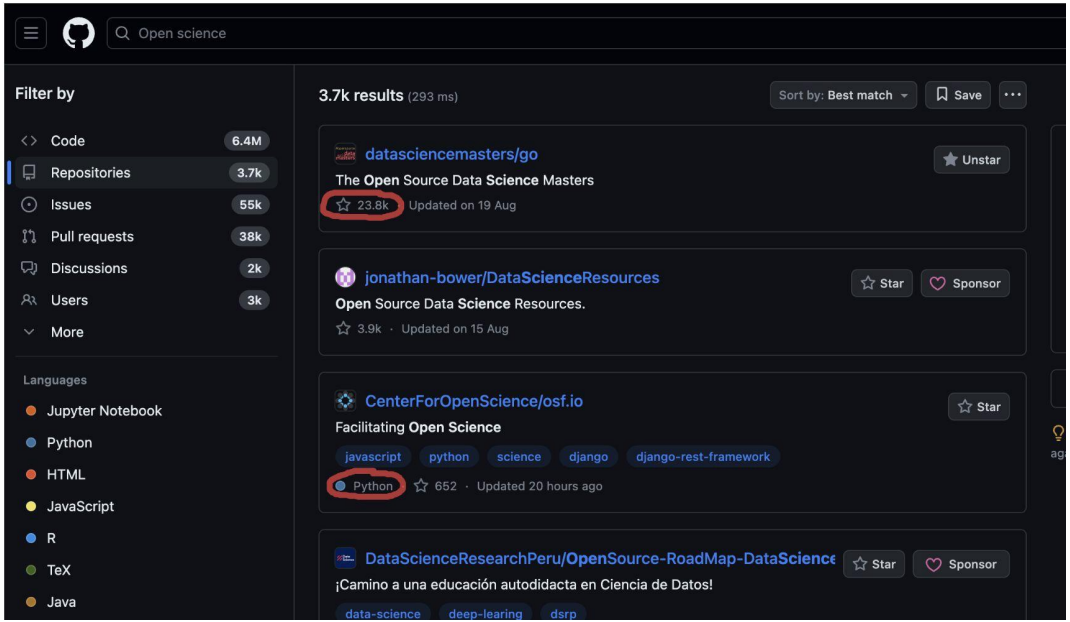


Getting to know the Github interface

Repositories on Github are divided into:

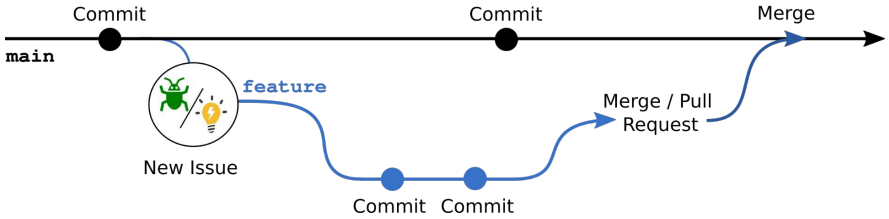
- Those that you have write access to
- Those you can only view (the great majority)

You can use the search function to query all public repositories on Github



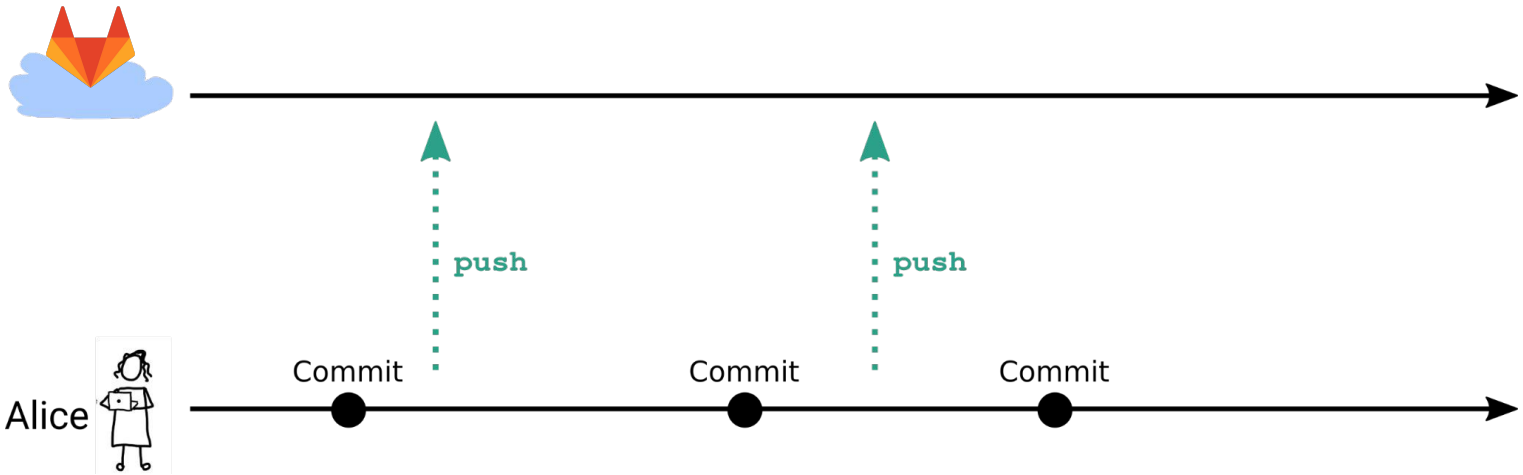
Collaboration with Git

Agenda

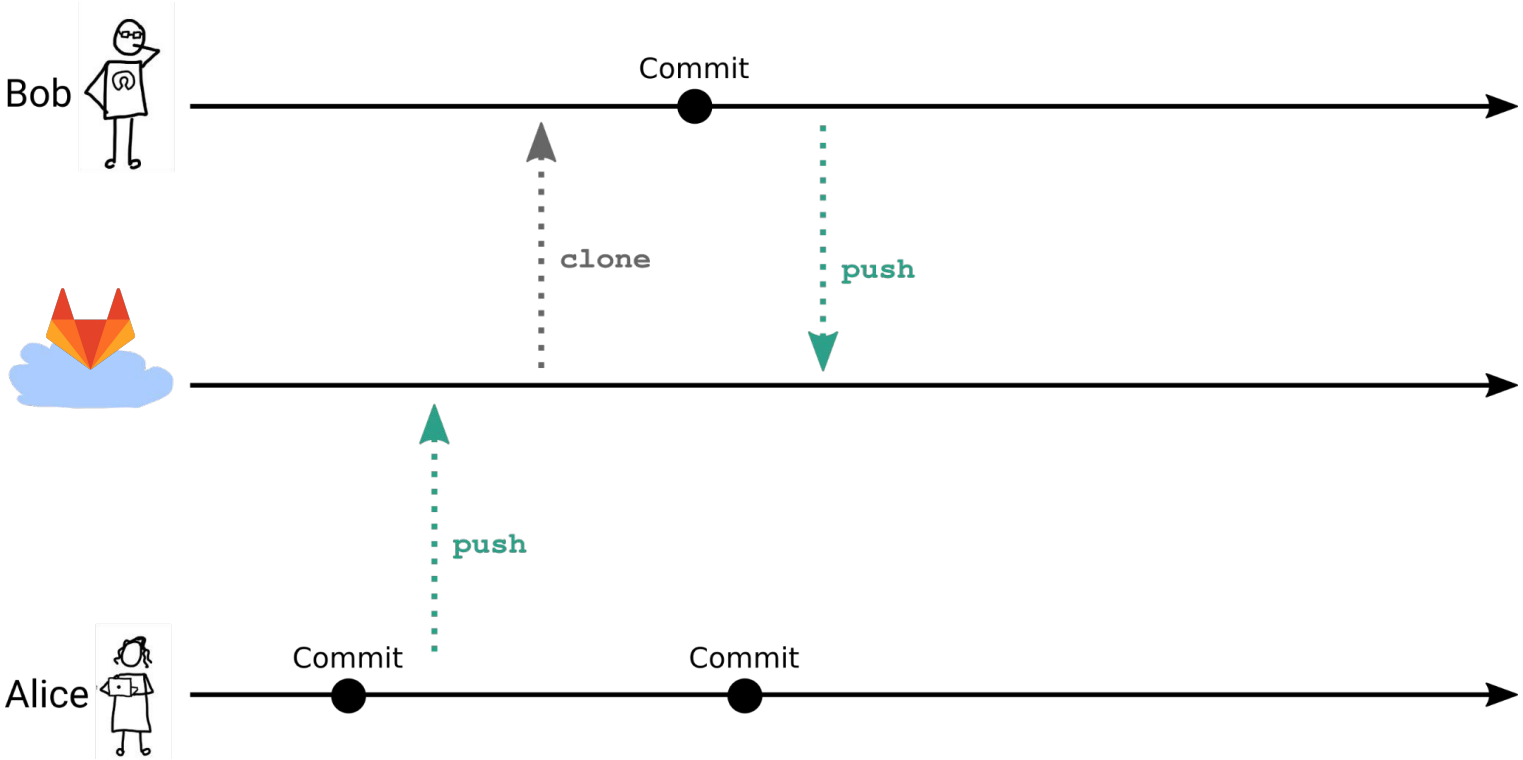


- Integrating changes from remote repository
- Learn GitLab's collaborative features
  - Manage repository access
  - How to create an Issue
  - Create a branch
- Merge branches …
  - using the command line
  - using GitLab

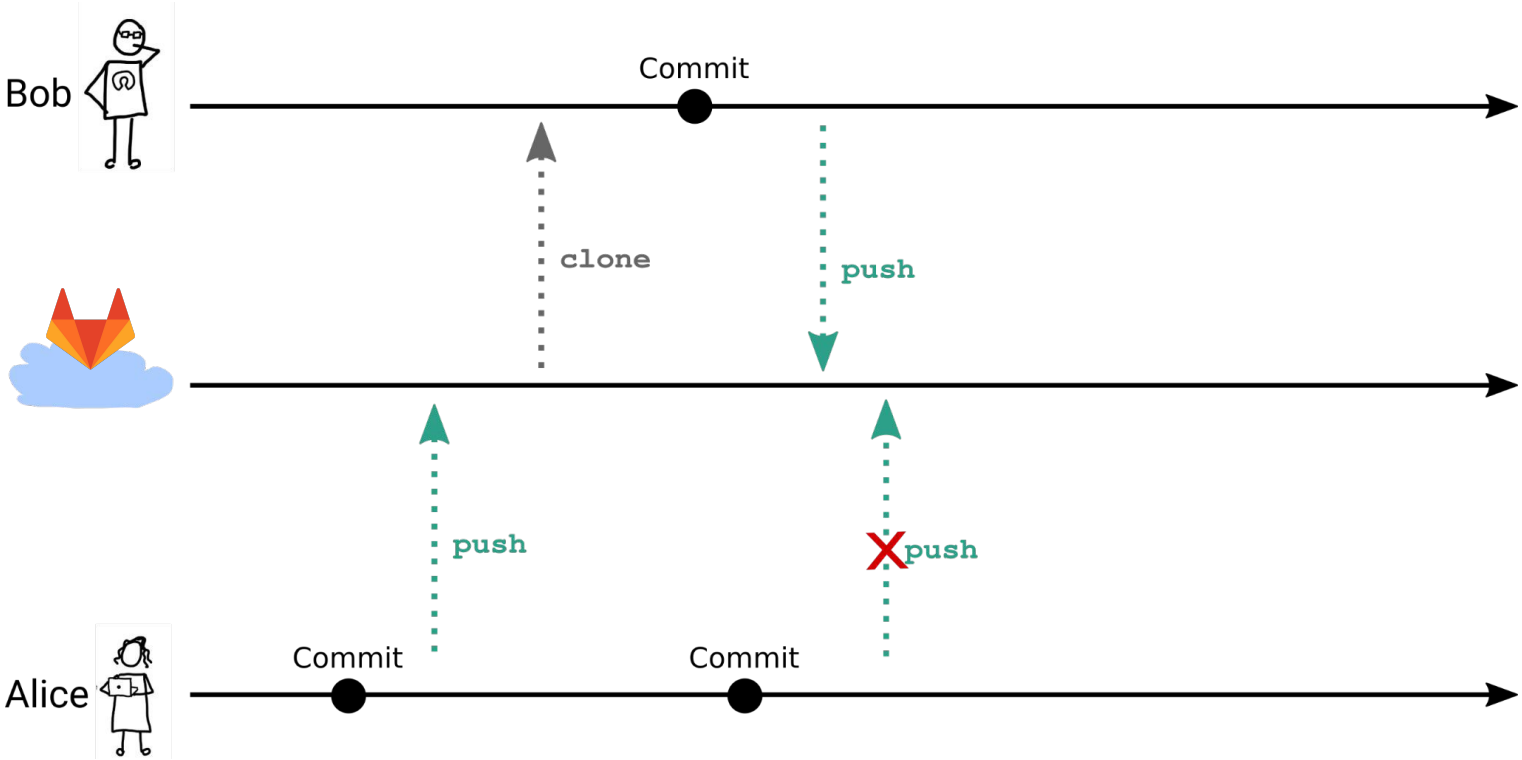
Collaborative Git workflow



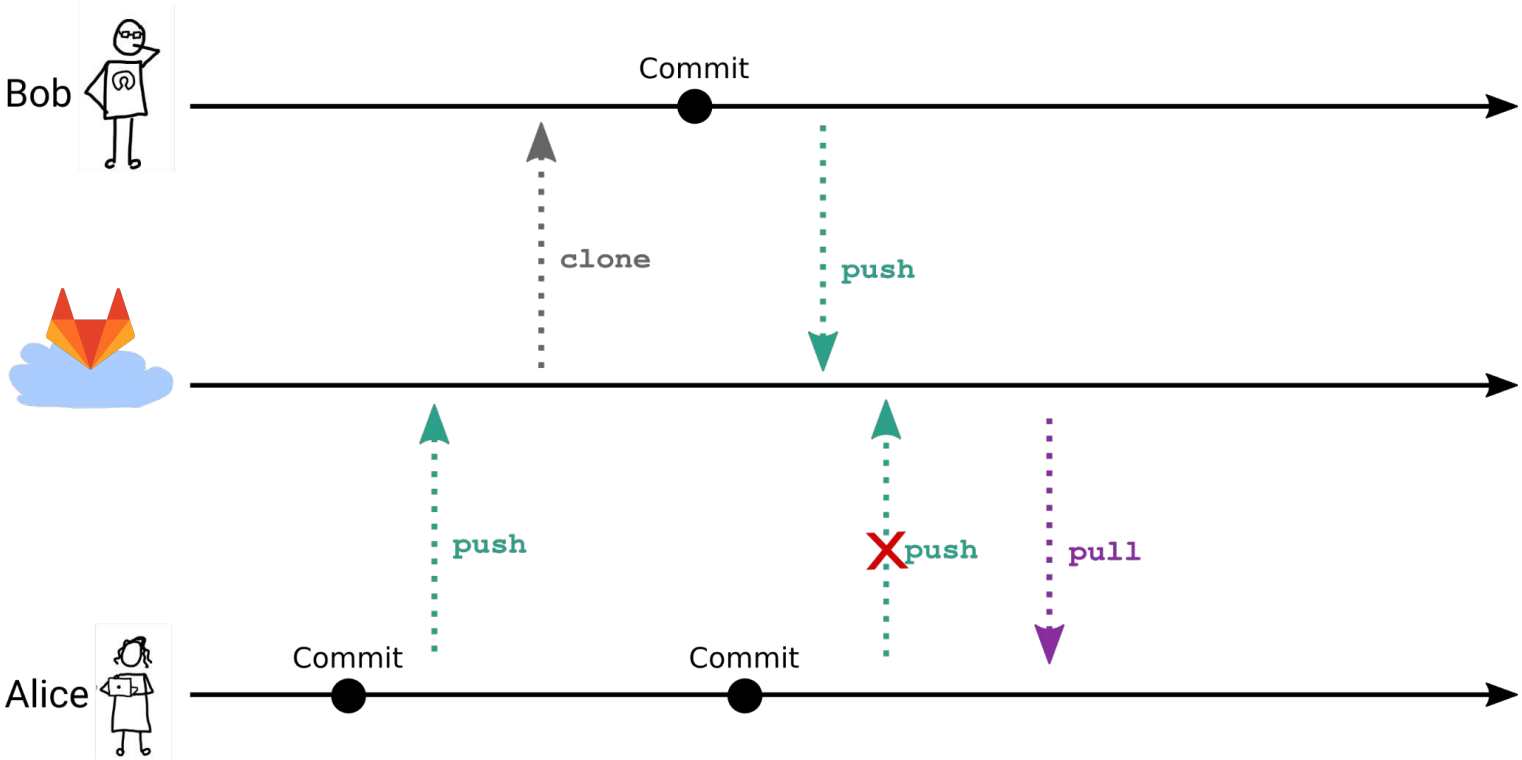
Collaborative Git workflow



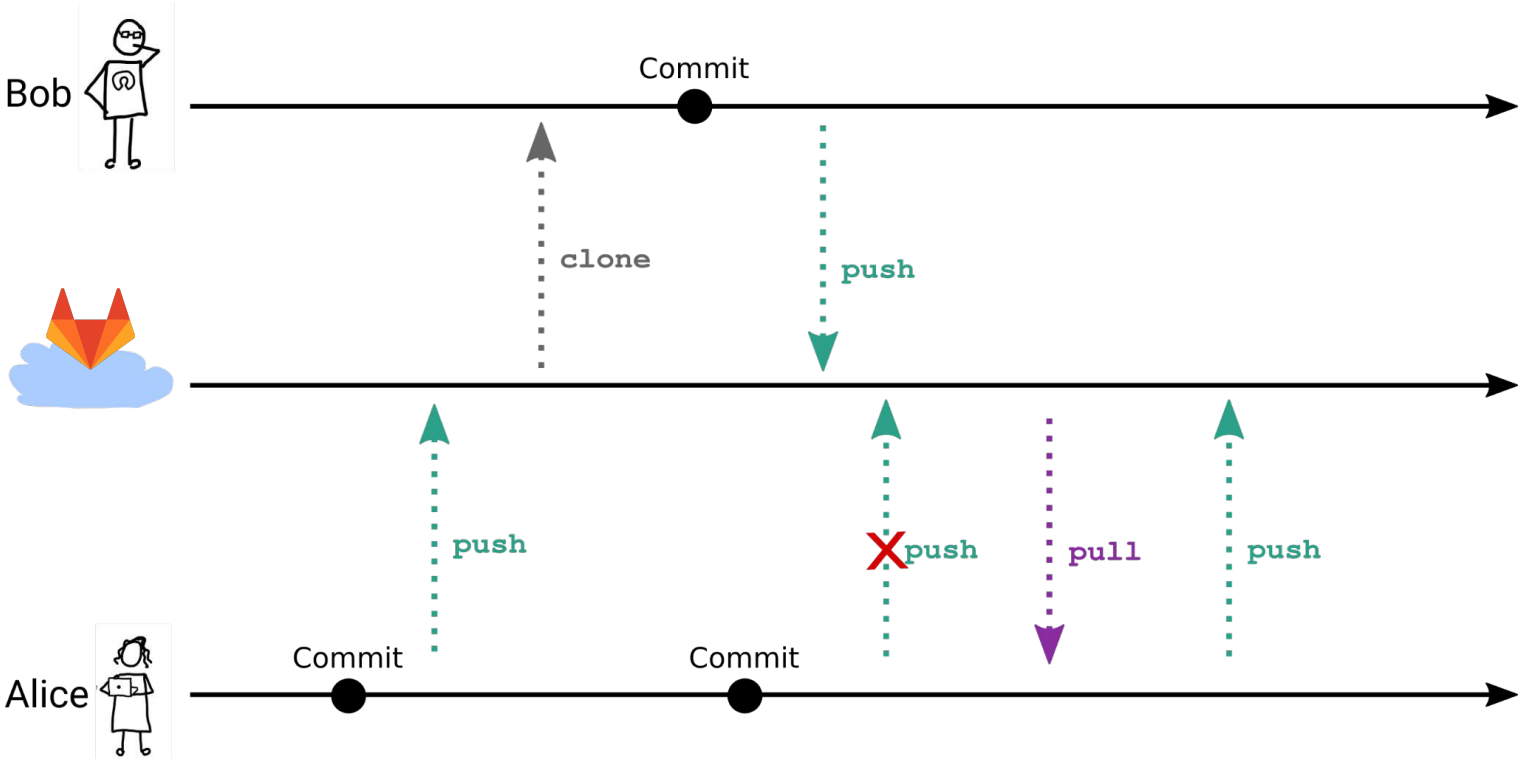
Collaborative Git workflow



Collaborative Git workflow



Collaborative Git workflow



Activity 1 (20’)

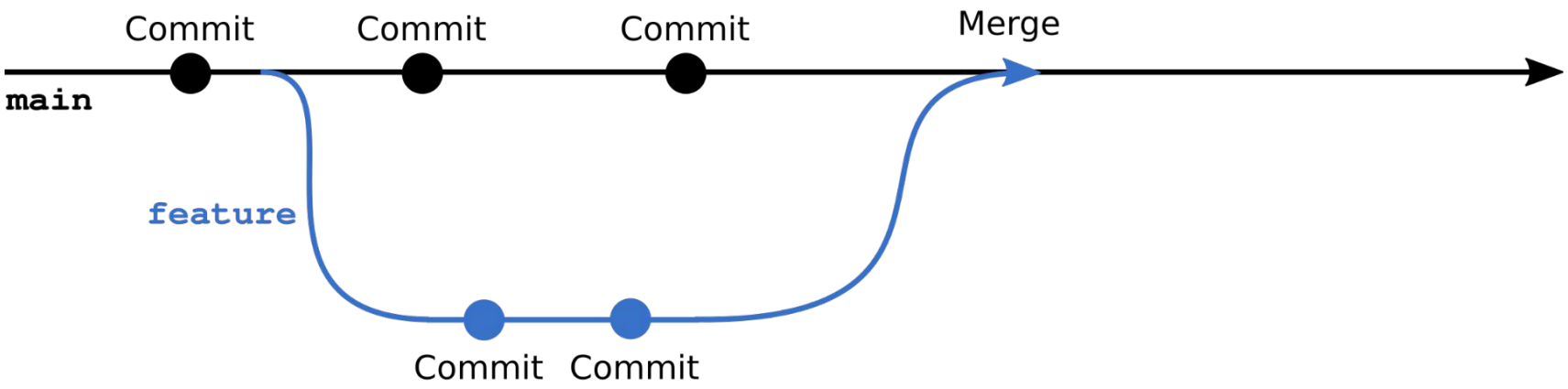
1.

Working on Github online

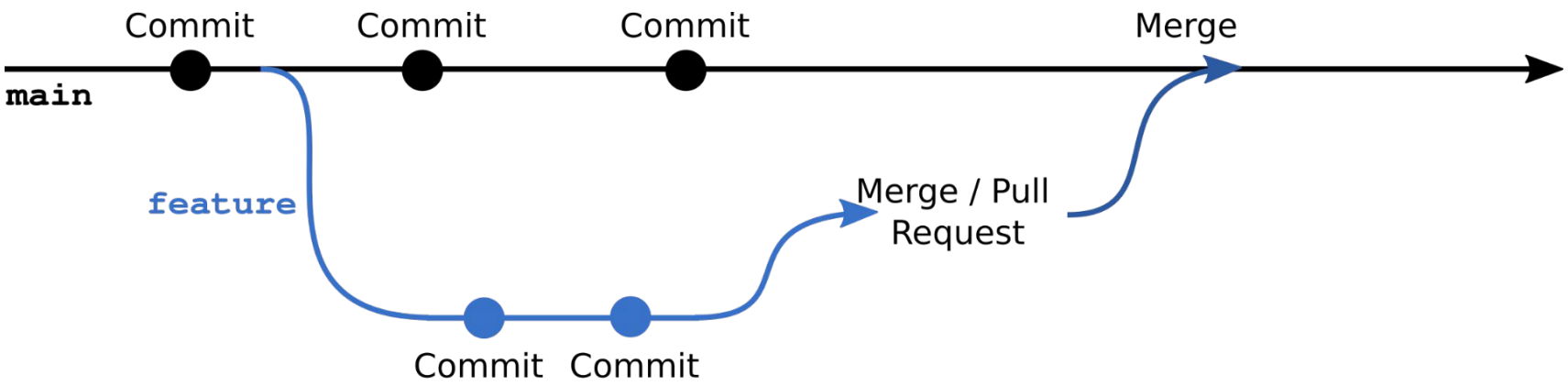
1. Log into your Github account.
2. Fork the course repo: `https://github.com/fritjoflammers/RSE_Juelich`
3. In the folder for `day1.1`, you’ll find a Readme.md.
4. Do Activity 1:
    a. Create a file, add your favourite animal and make a PR.

Branches

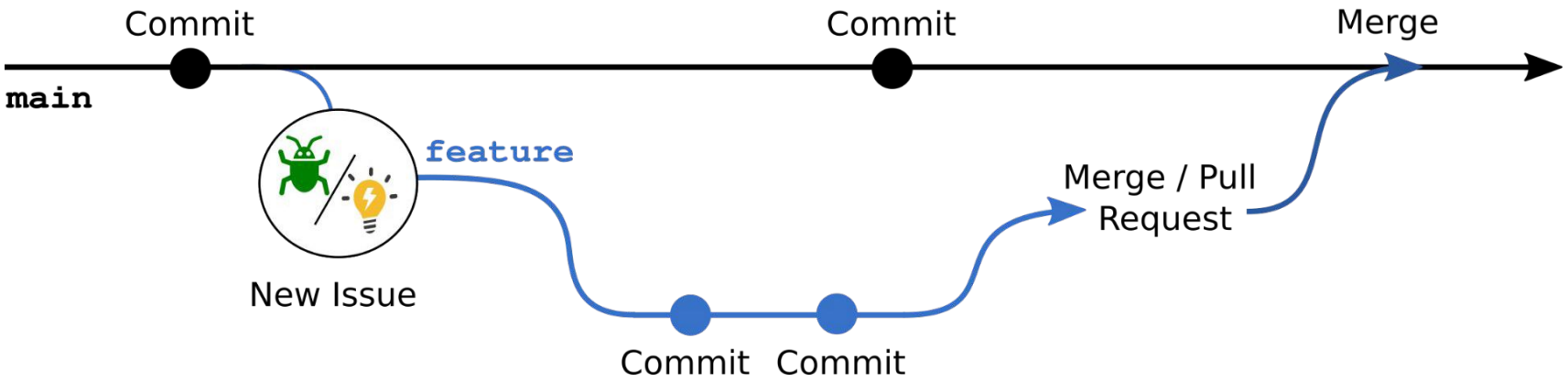
The main-feature branch workflow



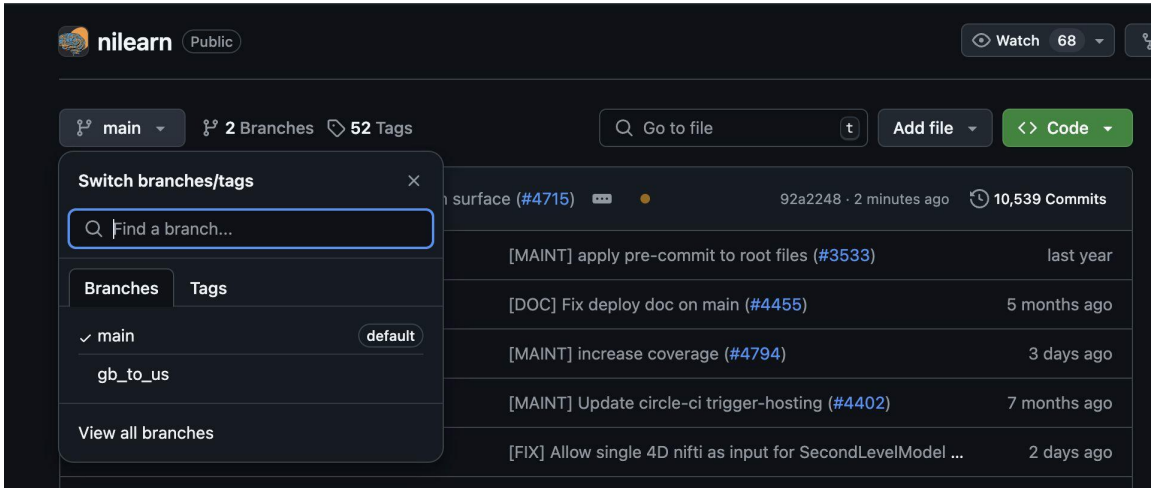
The main-feature branch workflow



A typical collaboration workflow



Make branch and merge back into your code



Activity 2 (15’)

Branching

1. Under the Readme.md section of day1.1, do Activity 2:
   Create a branch, make a change, merge it back into main

Summary: Recap terms

- **Git**
- **Github / Gitlab**
- **Commits**
- **Forks**
- **Branches**
- **Merge**

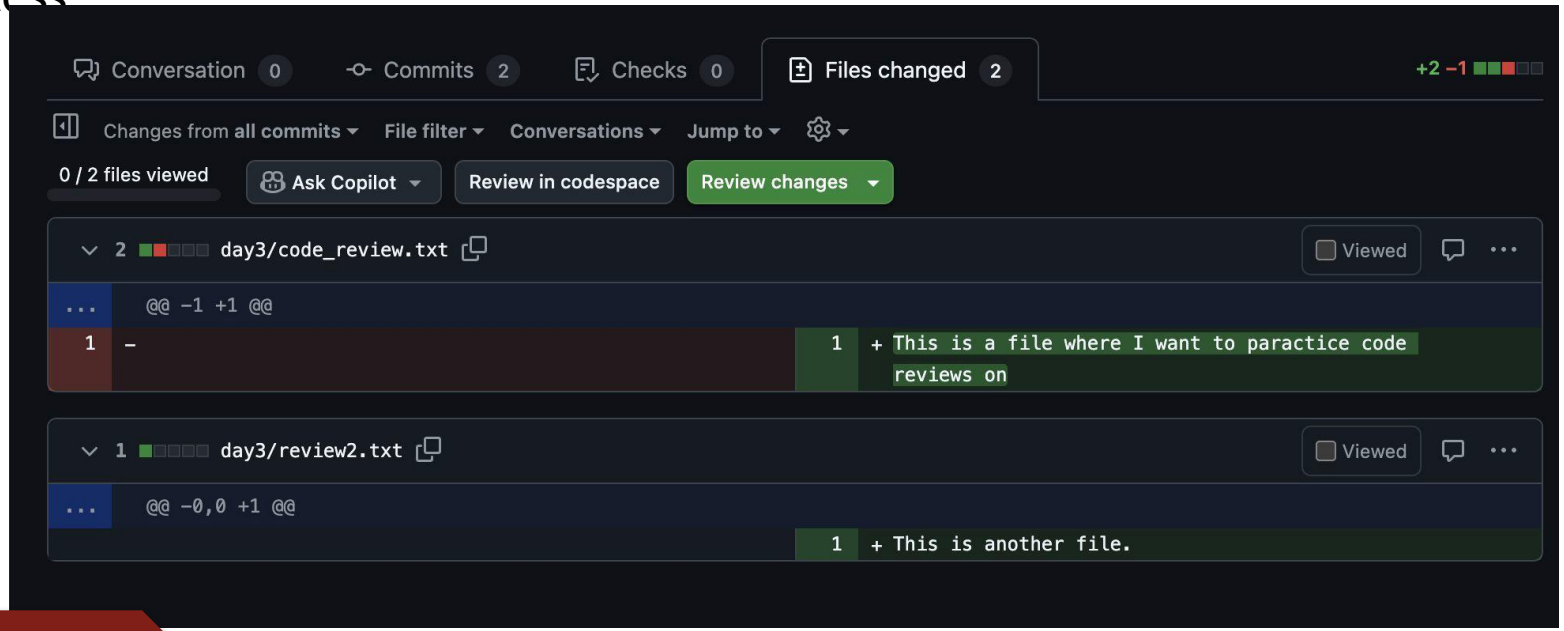
Summary

- **Git** is software providing decentralized version control system
- **Github / Gitlab** are web platforms that provide centralized access to repository and a graphical user interface
- **Commits** are the units of work that contain changes to the repository
- **Forks** are a copy of Github repository in your own account. They make a public, but read-only repository editable for you.
- **Branches** are diversions in the series of commits, allowing parallel work on the same codebase
- **Merge** integrates the changes of two branches into one

Break

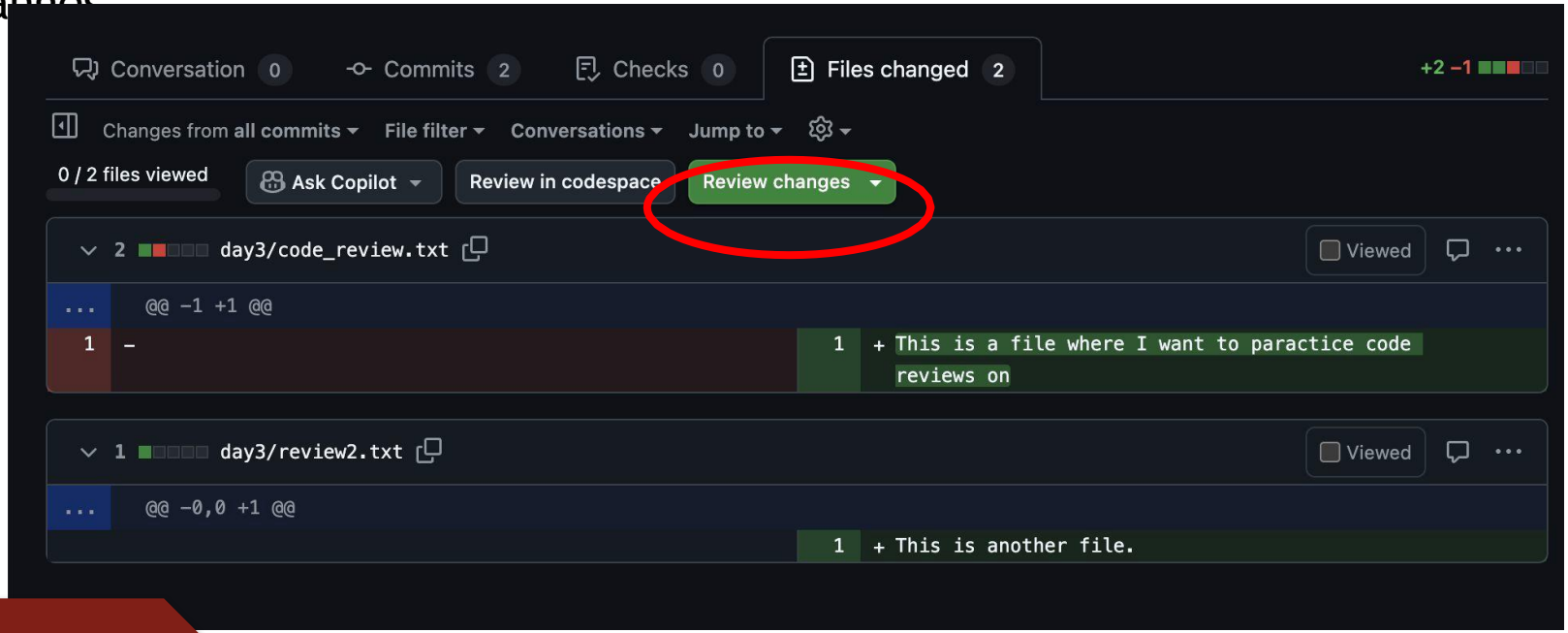
Review a PR online

Github allows for a very efficient code review process



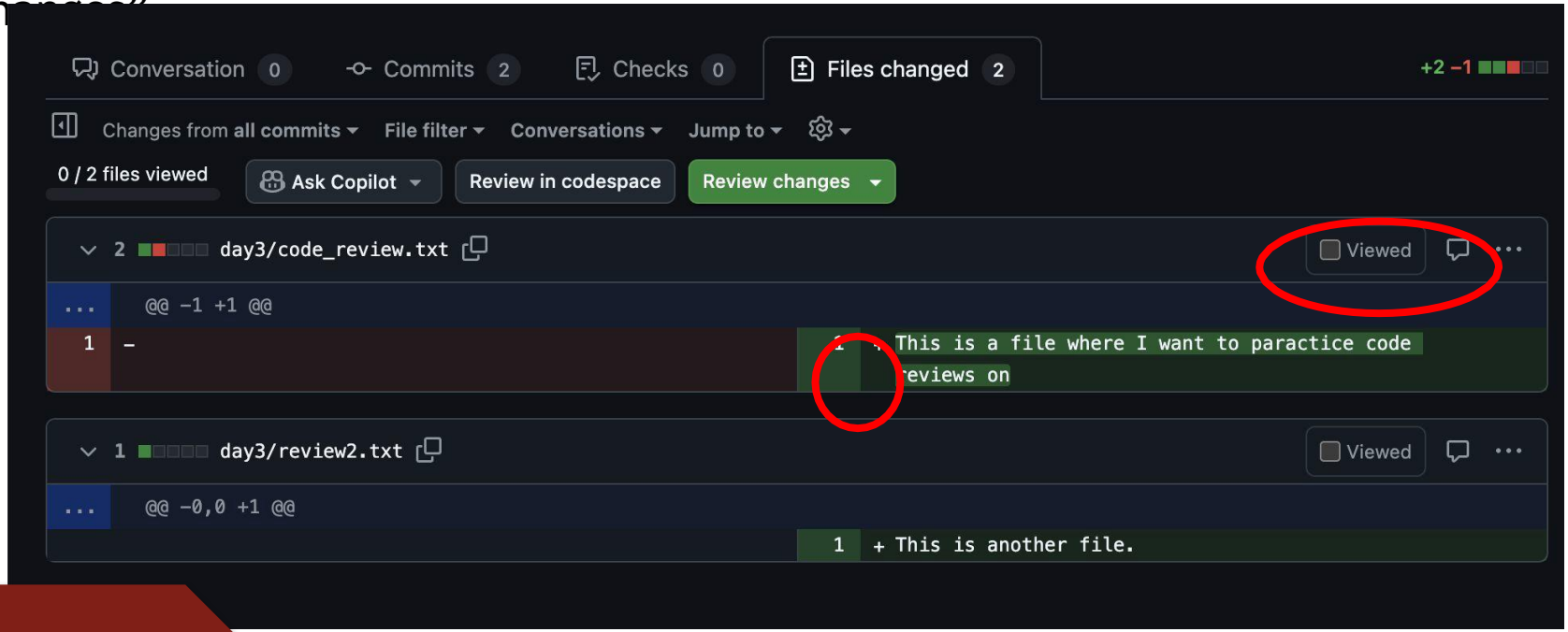
Review a PR online

Click on “review changes”



Review a PR online

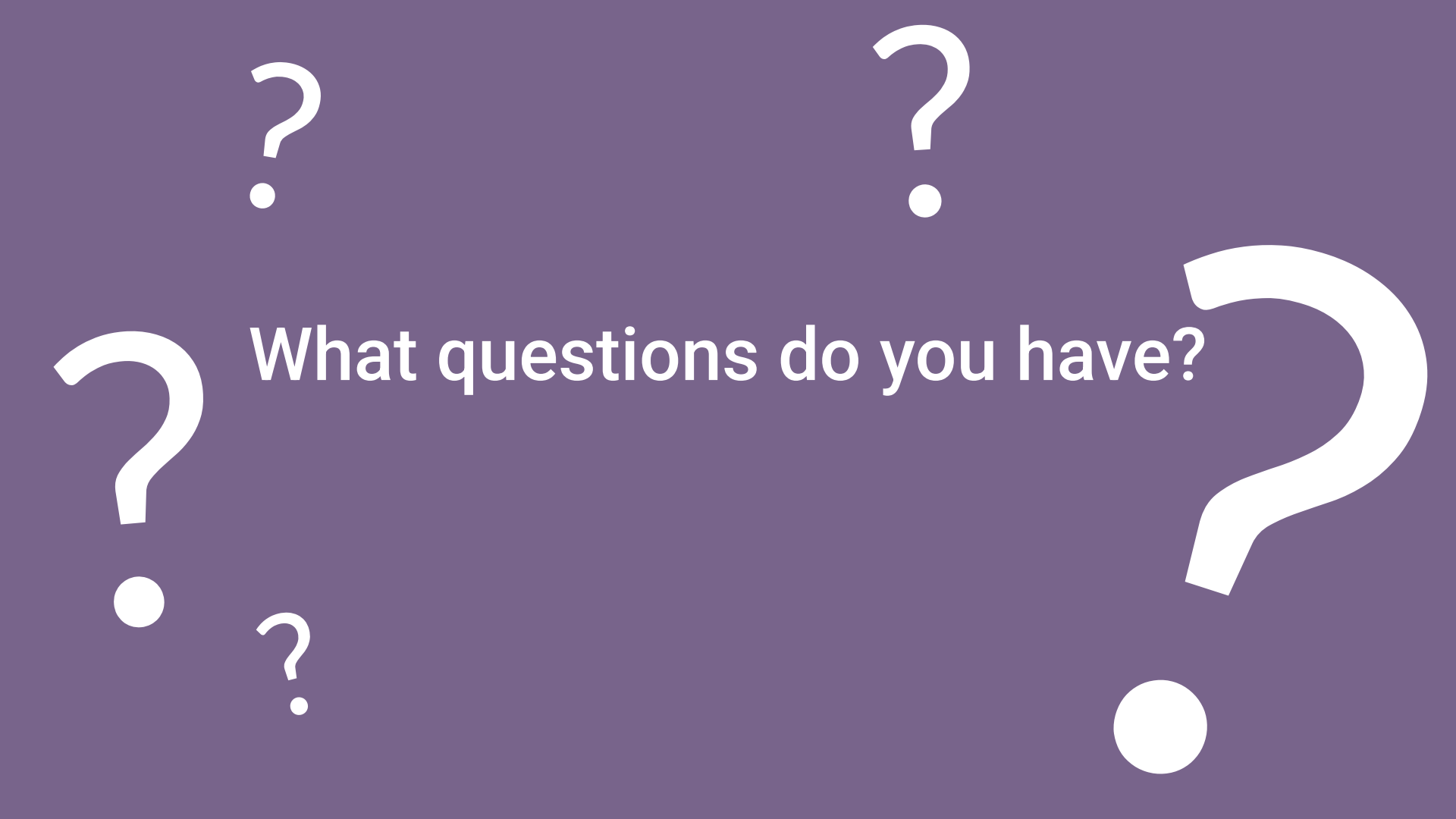
Click on “review changes”



Activity 3 (15’)
Forking, PR’s and code review

Under the Readme.md section of day1, do Activity 3

- Fork each other’s repo, make a PR, assign each other as reviewers and perform a code review


What questions do you have?

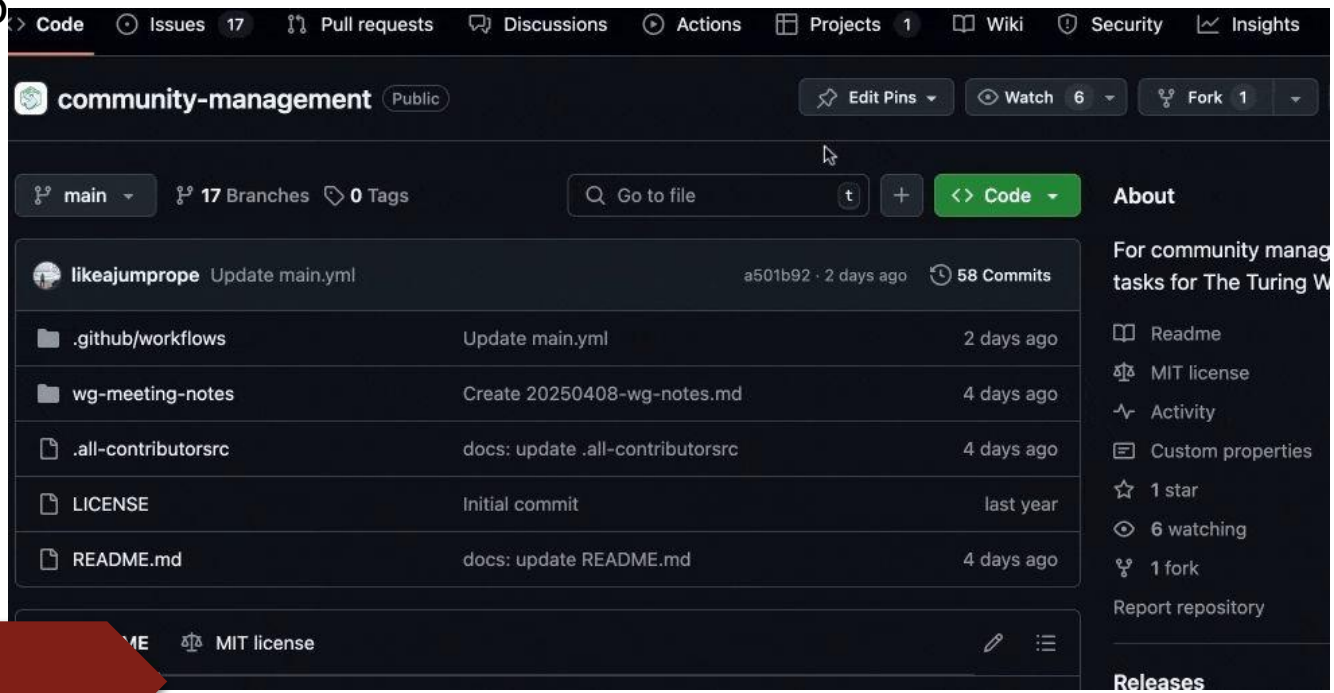
Summary

- **Git** is software providing decentralized version control system
- **Github / Gitlab** are web platforms that provide centralized access to repository and a graphical user interface
- **Commits** are the units of work that contain changes to the repository
- **Forks** are a copy of Github repository in your own account. They make a public, but read-only repository editable for you.
- **Branches** are diversions in the series of commits, allowing parallel work on the same codebase
- **Merge** integrates the changes of two branches into one
- **Pull requests** are a Github feature to propose changes, facilitate code review and merge changes into the codebase.
  Branch - Commit - Push - Pull request - Review - Merge

Some more Github features

Project management - boards on github

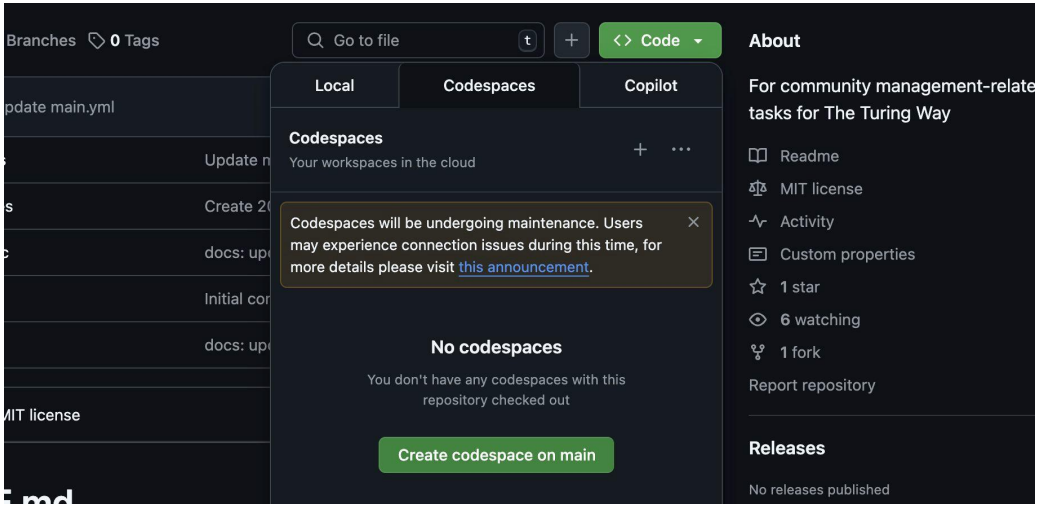
Project boards are a great way to organize a project and issues on github



Codespaces

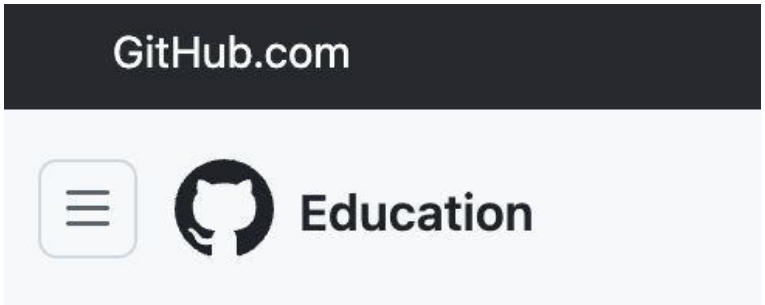
Github codespaces allows you to run files in Github repository in VS code instance in your browser.

- Good for quickly checking things
- CAREFUL: You have some free quota (as student etc), but then it's a paid service
- Close your instance!!



Github Education

- As a registered student, you can gain access to Github education
  - Free perks, such as:
    - Free Co-Pilot and Codespace
    - Full Notion AI plan
    - Licenses to Jetbrains IDE etc
    - Certifications



Github Copilot

- Chat-GPT for CODE
- Integratable with VS code

- If you are a registered maintainer of an open source repository, you also get free access to codespaces and co-pilot.

Ressources

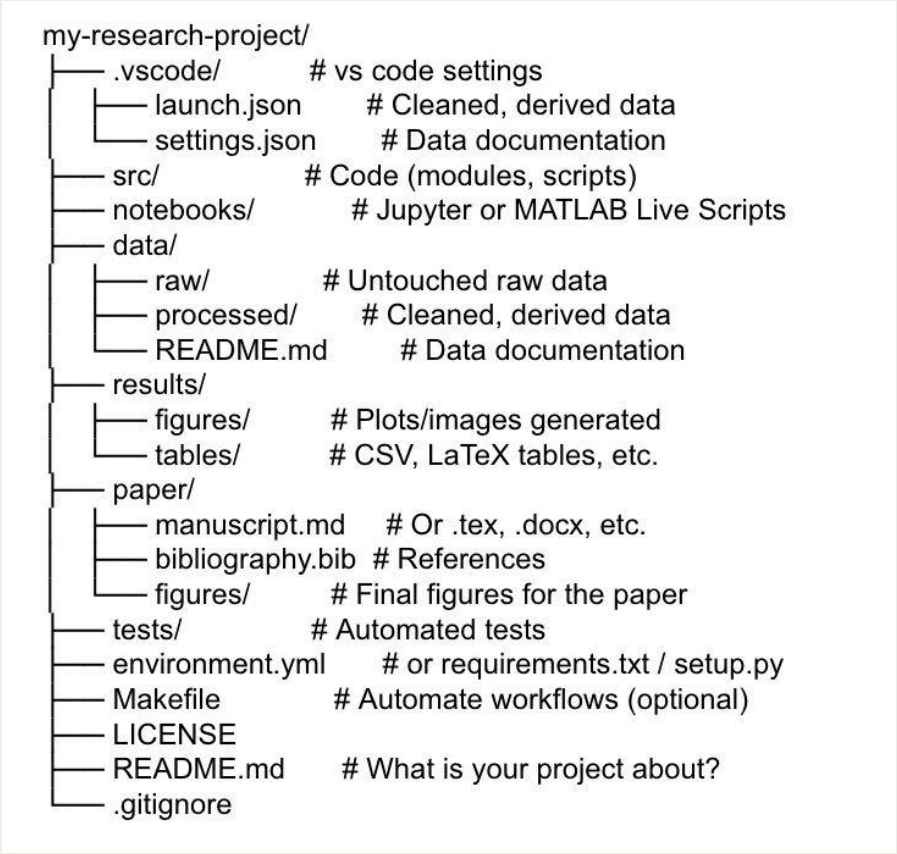
- Baxter, R, Chue Hong, N, Gorissen, D, Hetherington, J & Todorov, I 2012, 'The Research Software Engineer', Digital Research 2012, Oxford, United Kingdom, 10/09/12 - 12/09/12.

- The Missing Semester of Your CS Education: https://missing.csail.mit.edu/
- Nuest, D., Konkol, M., Pebesma, E., Kray, C., Schutzeichel, M., Przibytzin, H., & Lorenz, J. (2017). Opening the Publication Process with Executable Research Compendia. D-Lib Magazine, 23(1/2). 10.1045/january2017-nuest

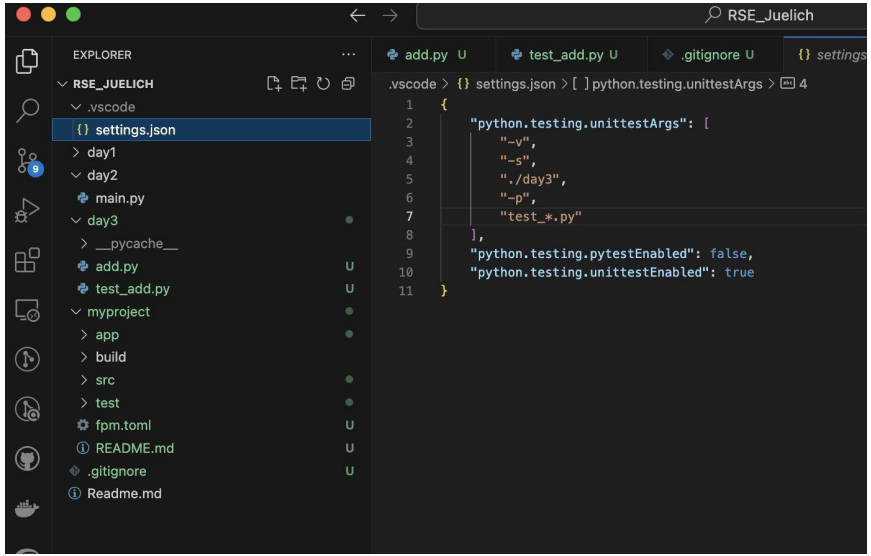
- BIDS standard: https://bids.neuroimaging.io/index.htm
- Goth, F. et al. Foundational competencies and responsibilities of a Research Software Engineer. F1000Res. 13, 1429 (2024).
- https://book.the-turing-way.org/collaboration/github-novice
- https://book.the-turing-way.org/reproducible-research/vcs
- https://fritjoflammers.github.io/dra-docker-training/learning-git/02-git_collaboration.html

Lunch

Your code editor



Setting up your code editor



- VS code
  - Get the extensions for you code base

Important files:

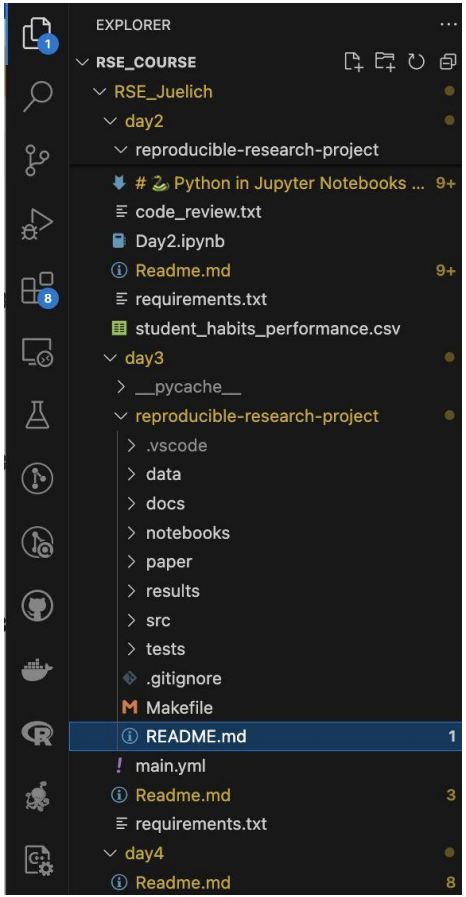
- Settings.json
  - Sets the behaviour of your vscode (linting, formatting etc)
- Launch.json
  - Specifies the behaviour of vs code running you programs
    - Debugging, which interpreter, which language

Important VS code extensions

Extensions for your programming language;

- Python, Julia, etc IDE.

- GitLens
- (Docker)
- Jupyter notebook extension



settings.json

Purpose: Configure **editor behavior**, **extensions**, and general **environment settings**

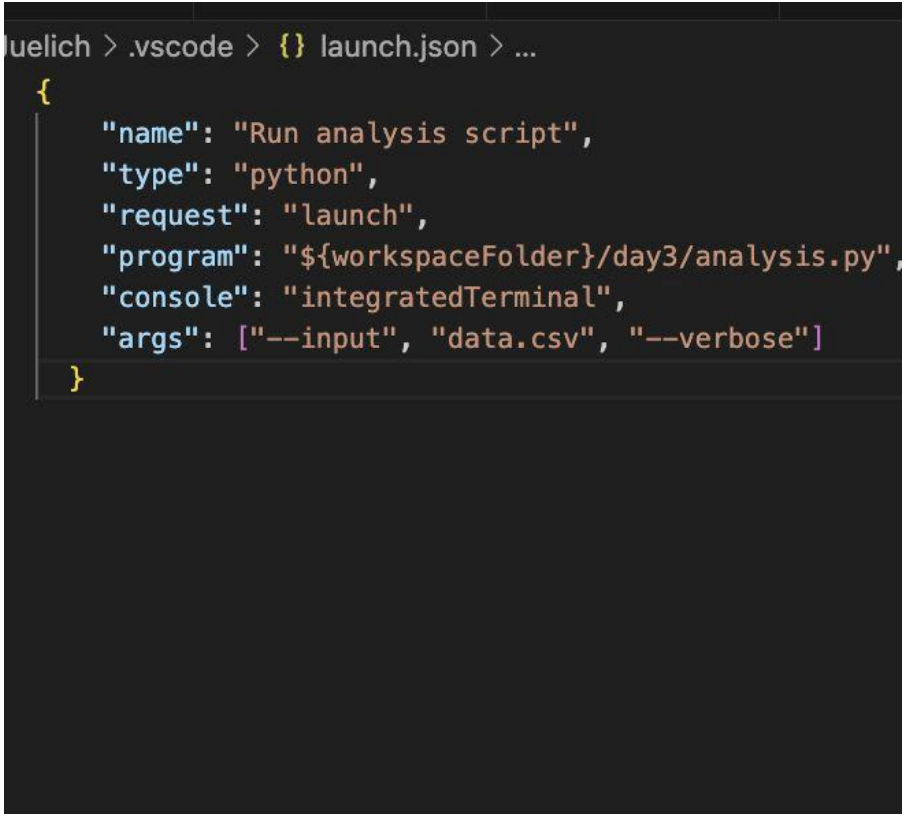
Scope: User-wide or workspace-specific
Examples:

- Tab size, font, auto-save, linting rules
- Default Python interpreter or Fortran compiler path
- Format-on-save, excluded files, theme

launch.json

Specifies:

- Which script or executable to run
- What **command-line arguments** to pass
- Whether to **launch** a new process or **attach** to an existing one
- Where to show the output (terminal or debug console)
- Environment variables, directory, debugging ur

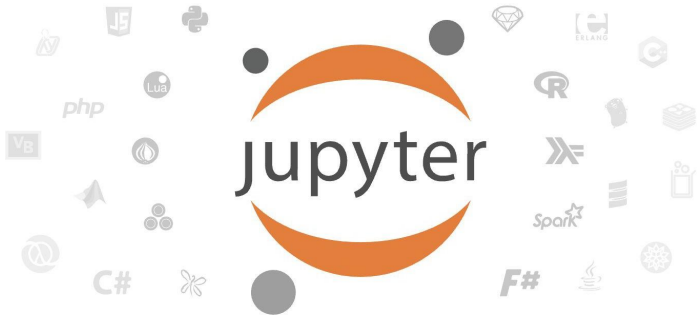


Jupyter notebooks

- Can be installed via `pip install notebook` and the launched via `jupyter notebook notebook.ipynb` or `jupyter notebook`
- Allows to run notebooks in the browser
- Not super great for large research projects, but:
  - Operating system agnostic
  - Can be shared and opened in the browser via Google Colab
  - Can be run via vscode
  - A variety of kernels available

For research:

- Better: https://marimo.io/



Matlab environment

Matlab projects

Use MATLAB Projects to manage files, paths, and settings.

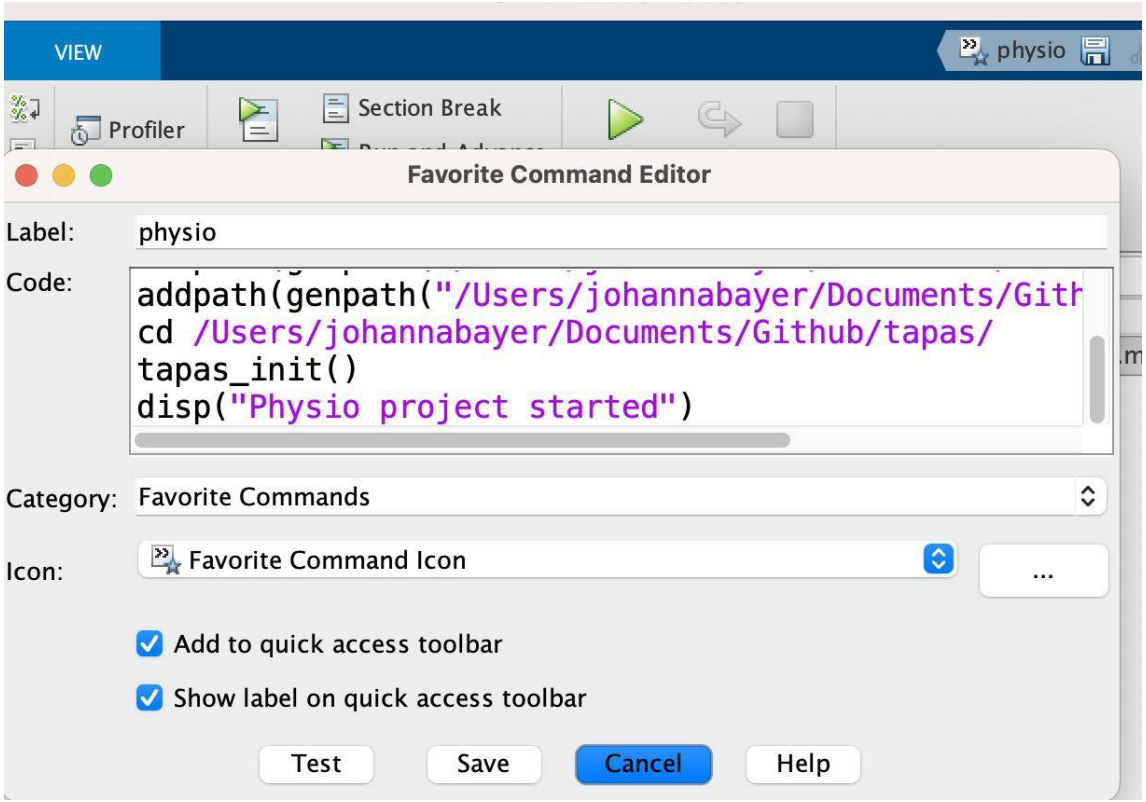
- Go to **Home > New > Project > From Folder**
- Choose your folder (e.g., your project-name/ directory)
- MATLAB creates a .prj file to manage paths, startup code, etc.
-

Configure startup paths:

- **Add a startup.m file** to automatically configure your environment on project open:

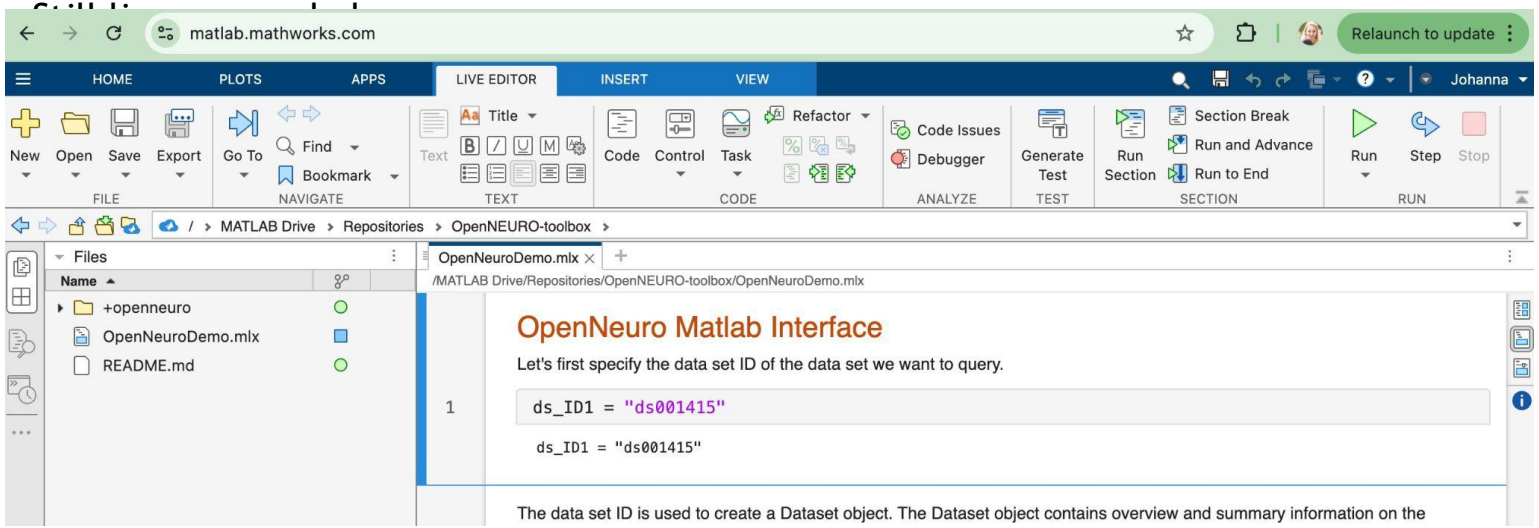
```
% startup.m
addpath(genpath('src'));
addpath('scripts');
disp('Project environment initialized.');
```

Set favourite commands



Matlab Live Scripts

- Cell based matlab scripts (.mlx)
- Can be opened in the browser (Matlab online)
  - Allows to share and execute code
  -



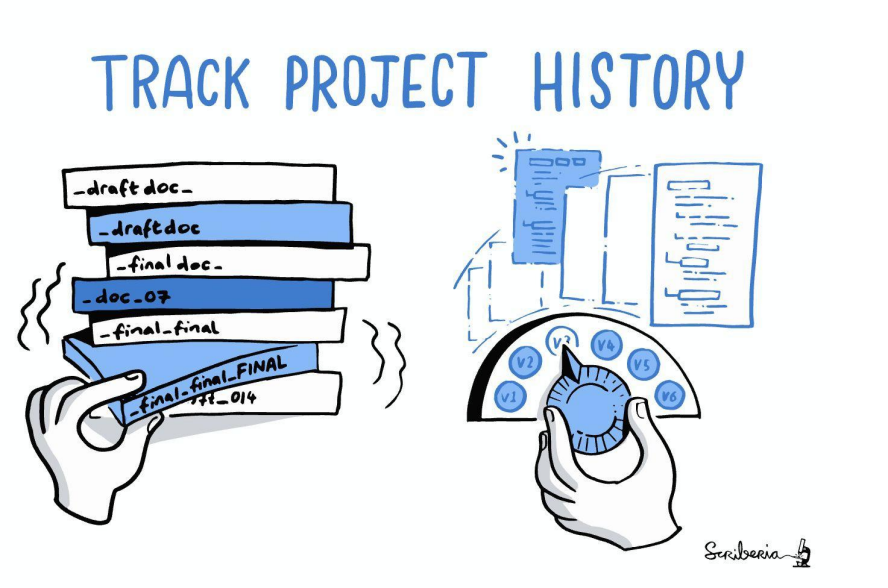
Set up how to compile/run your code

- Run via VS code
- Run via the command line/terminal
- Run via Jupyter notebook
- Run via a (other) IDE
- Run on a cluster/cloud …

Setting up a coding project on a cluster

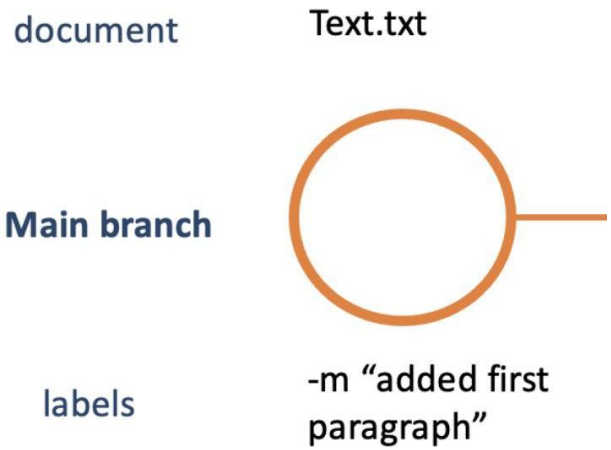
- Rsync of folders between cluster and local (images, tables etc)
- Ssh into cluster via vscode
- Sbatch

Committing



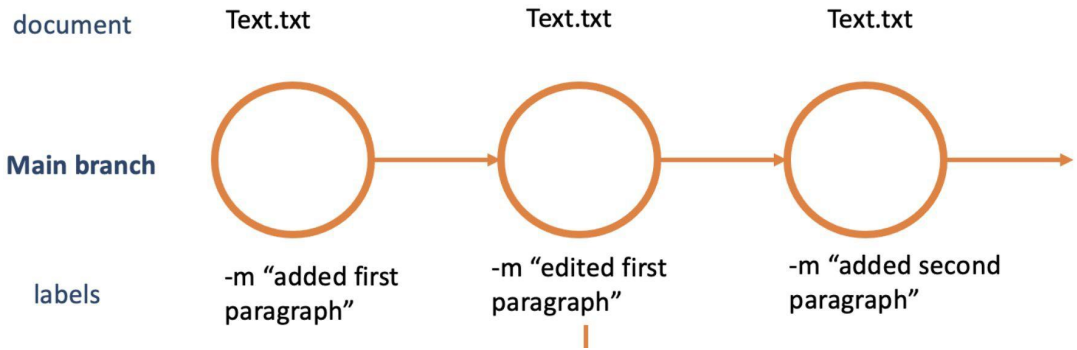
What is a commit?

- **Commits:** annotated snapshot of changes in a document



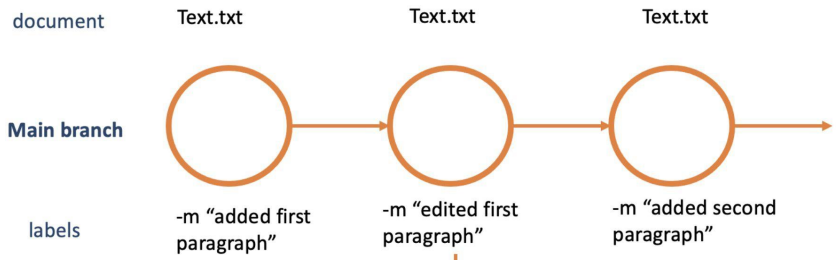
What is a commit?

- **Commits:** annotated snapshot of changes in a document

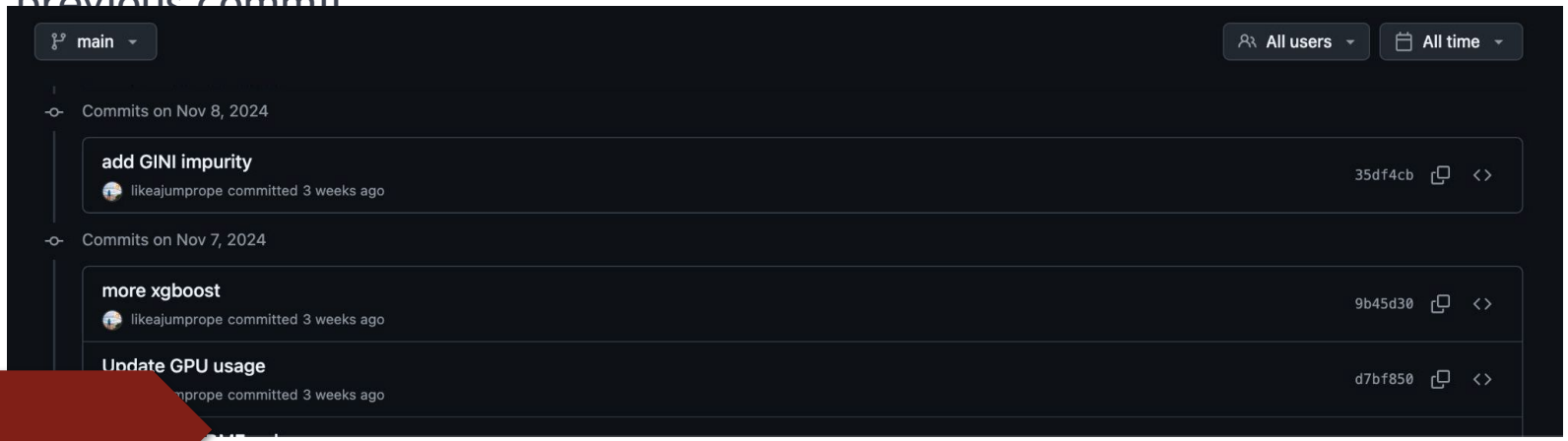


Git and Github: tracking your files, scripts and code

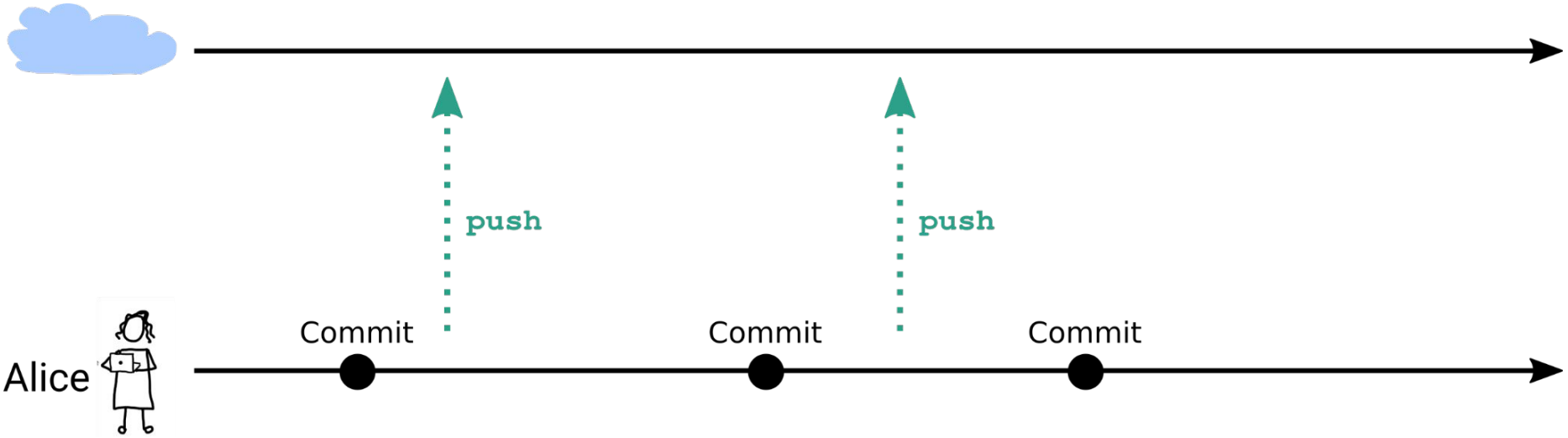
- Version control for files and scripts



- Ability to revert back to a previous commit

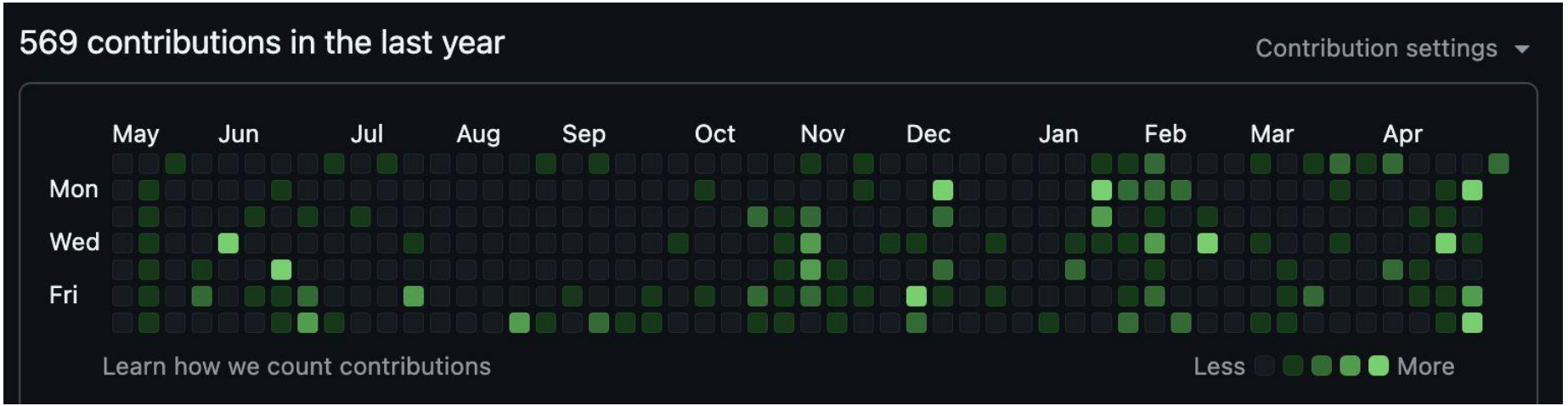


Git workflow for a single developer



Git and Github: tracking your files, scripts and code

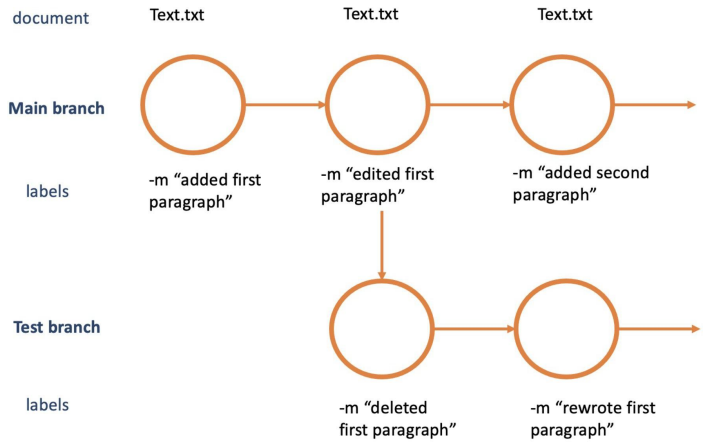
- Commit history



Git and Github: tracking your files, scripts and code

Collaboration (github online)

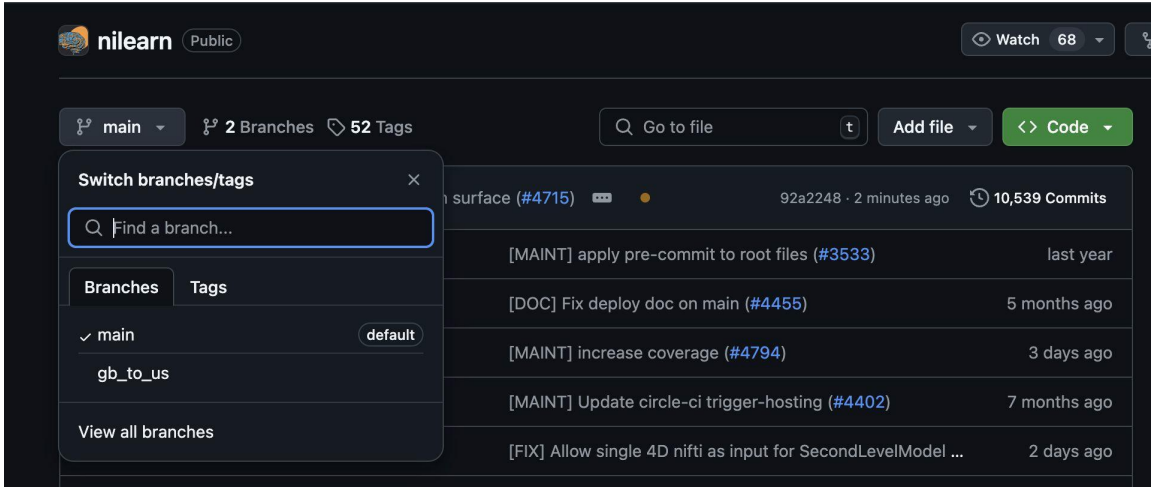
- **Branch**: Diversion point of repository and history



Git and Github: tracking your files, scripts and code

Collaboration (github online)

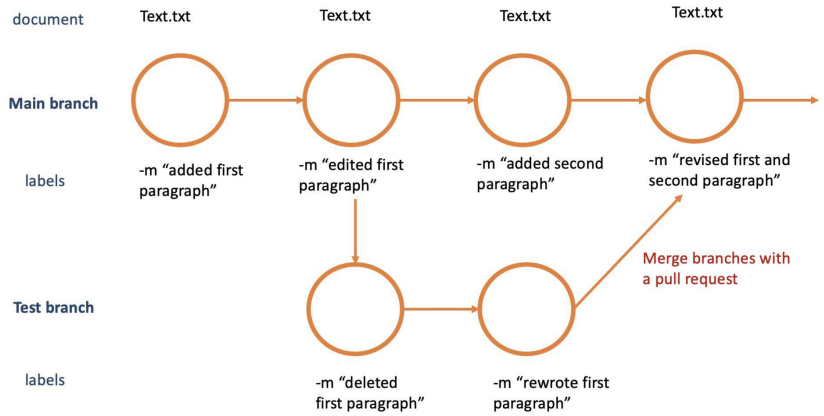
- **Branch**: Diversion point of repository and history



Git and Github: tracking your files, scripts and code

Collaboration (github online)

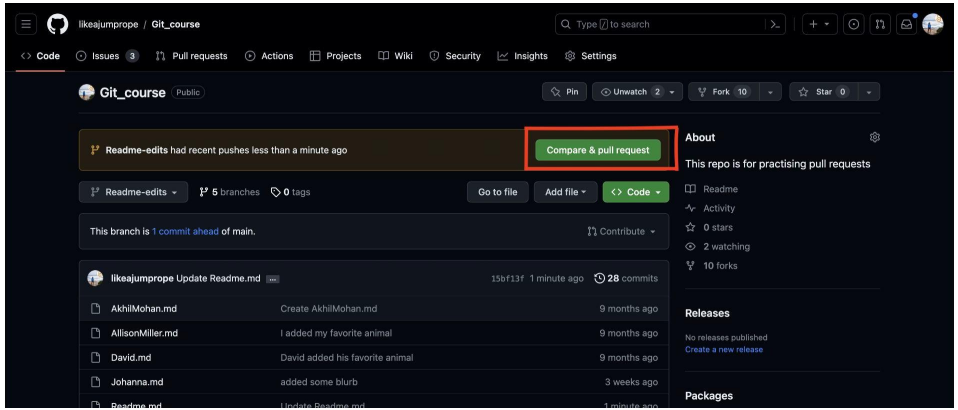
- **merging:** Unifying divergent branches, integrating changes
- Merging between branches is performed using a **pull request.**



Git and Github: tracking your files, scripts and code

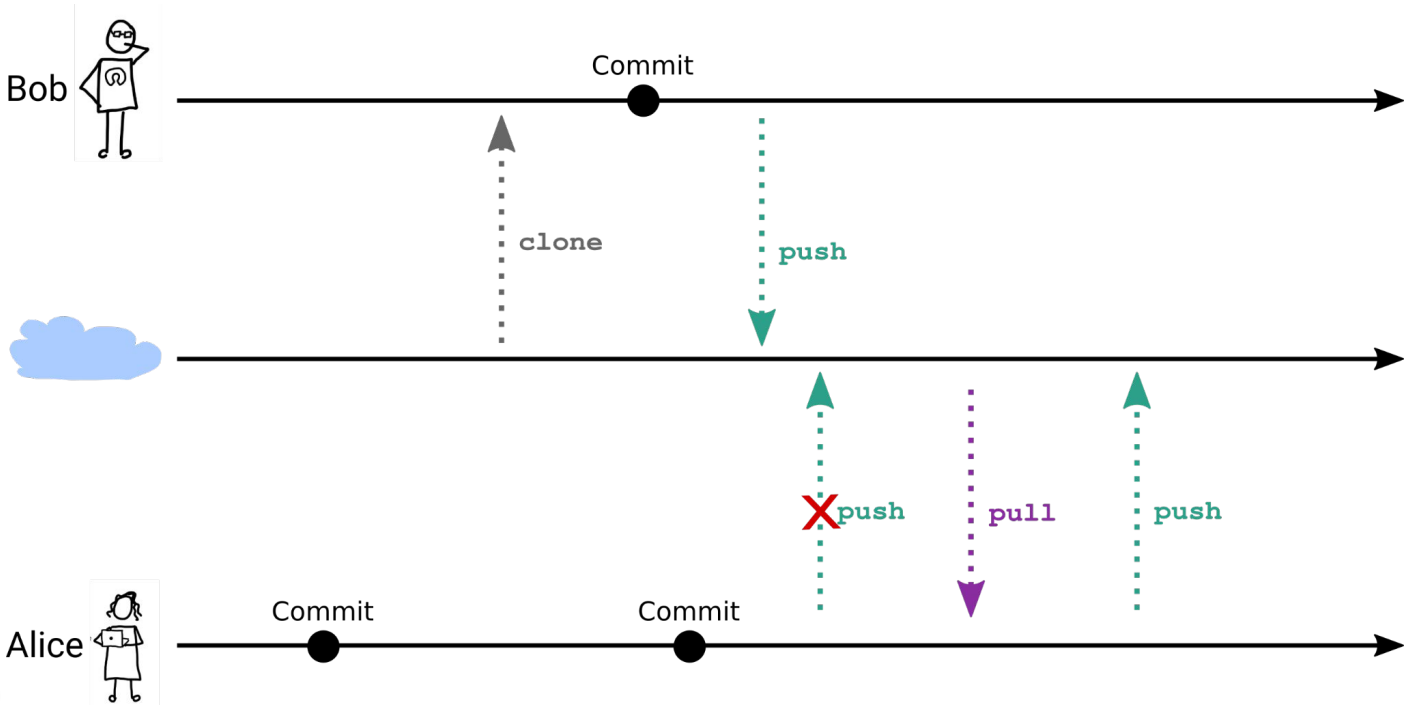
Collaboration (github online)

- **merging:** Unifying divergent branches, integrating changes
- Merging between branches is performed using a **pull request.**



Git and Github: tracking your files, scripts and code

Collaboration (github online)



Git and Github: tracking your files, scripts and code

Collaboration (github online)

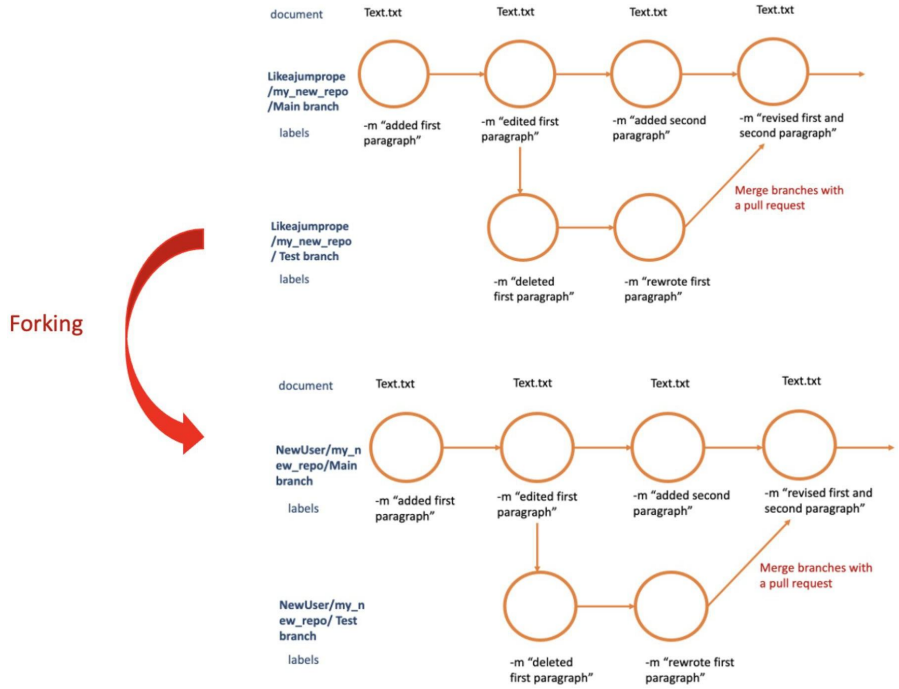
- **Fork**: make a copy of someone else's repository under your account.
- Repositories on Github are divided into two categories
  - Yours (you have writing permission)
  - Other people's (you don't)

- **Forking** creates a copy of someone else's repository that you own (aka you have writing permission)

Git and Github: tracking your files, scripts and code

Collaboration (github online)

-

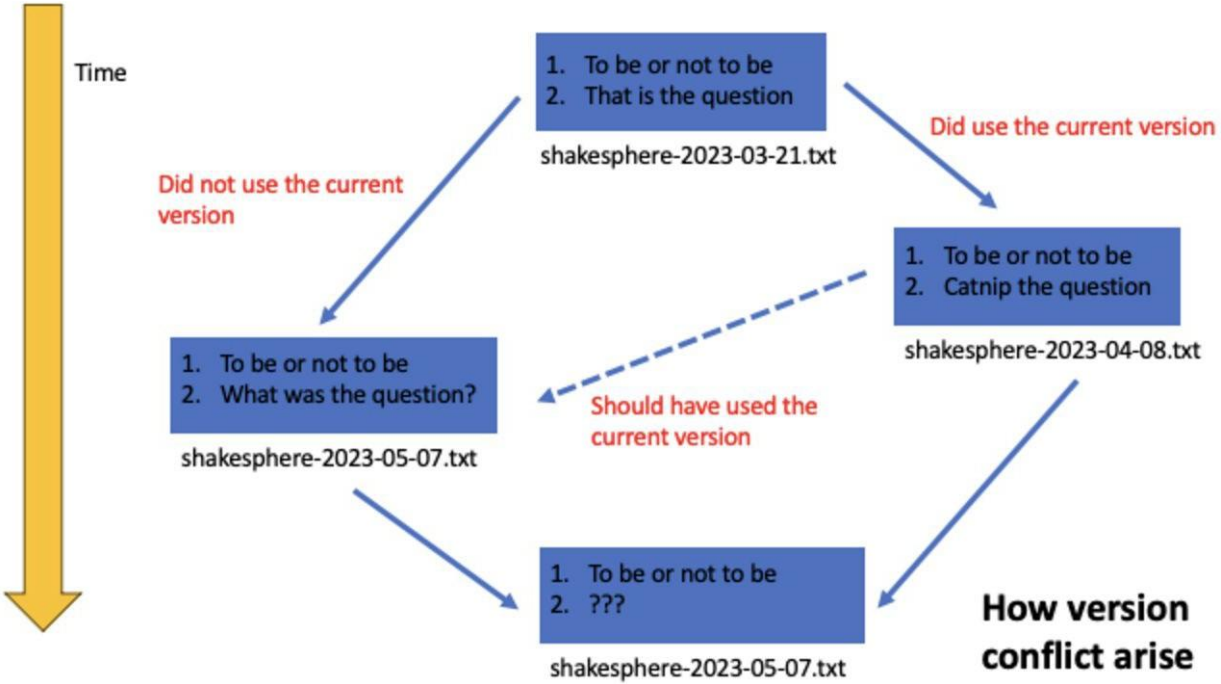


Why is it called pull request?

- Remember the separation between your and other people's repositories.
- In order to get your changes into someone else's repository, you have to request for them to be pulled in. This action is not up to you.
- You request a pull.
- The other person accepts, and pulls.

Fig. 191 Making your first pull request on GitHub. The Turing Way project illustration by Scriberia. Used under a CC-BY 4.0 licence. DOI: 10.5281/zenodo.3332807.

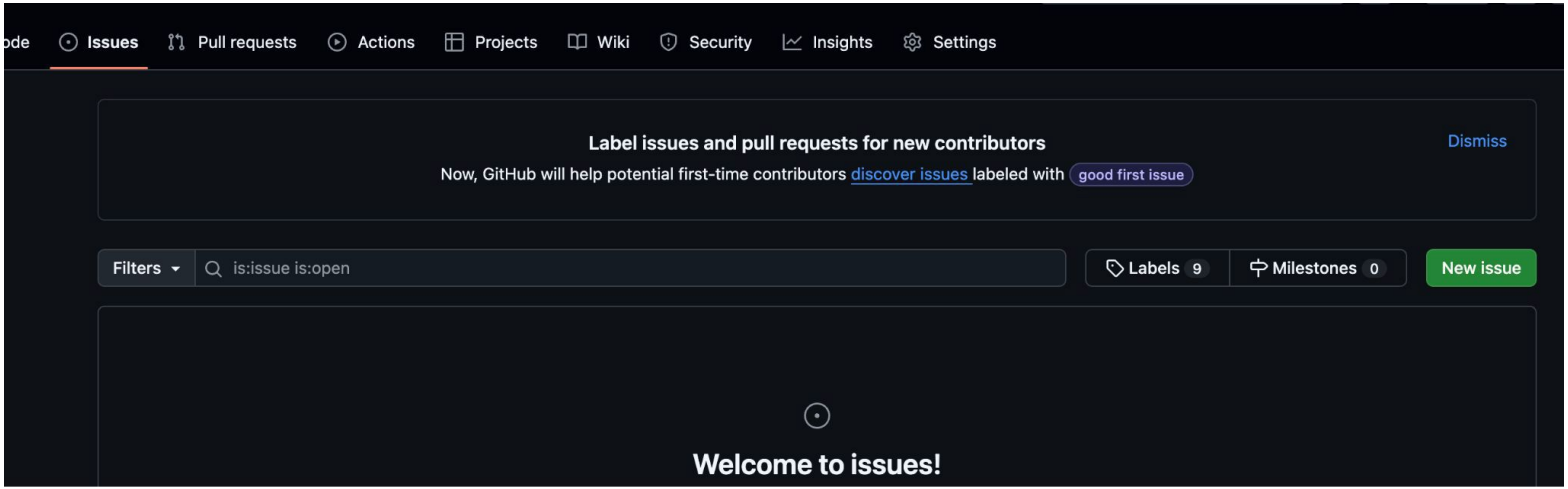
Merge conflicts



Resolving a merge conflict

Issues on Github

Issues on GitHub are a built-in tool for tracking tasks, enhancements, bugs, and other project-related activities. They provide a way for developers, collaborators, and even external users to communicate and manage the work associated with a GitHub repository.



Issues and collaboration on Github online

Issues are the way of communicating on Github

- Issues follow markdown format
- PRs can be referenced in Issues (and vice versa)
- Usually one PR responds to an issue